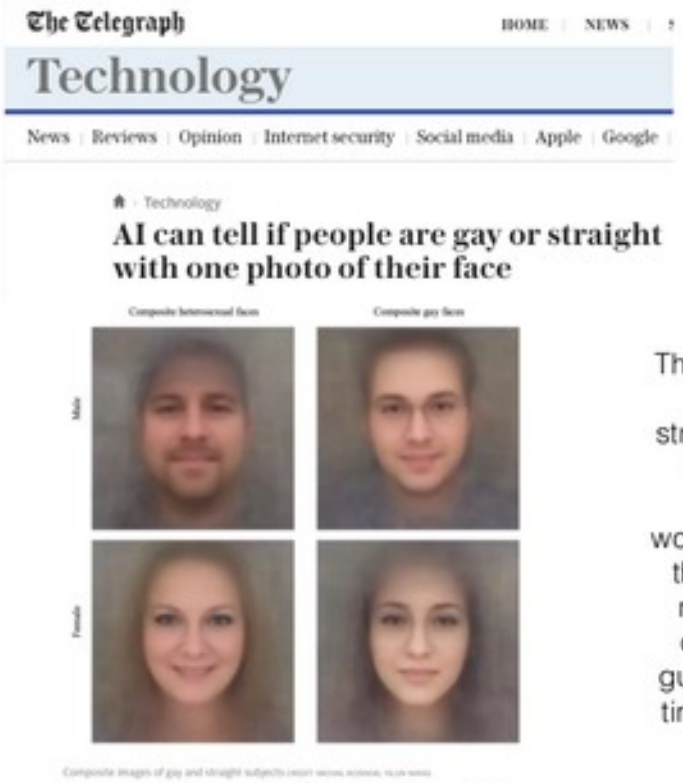
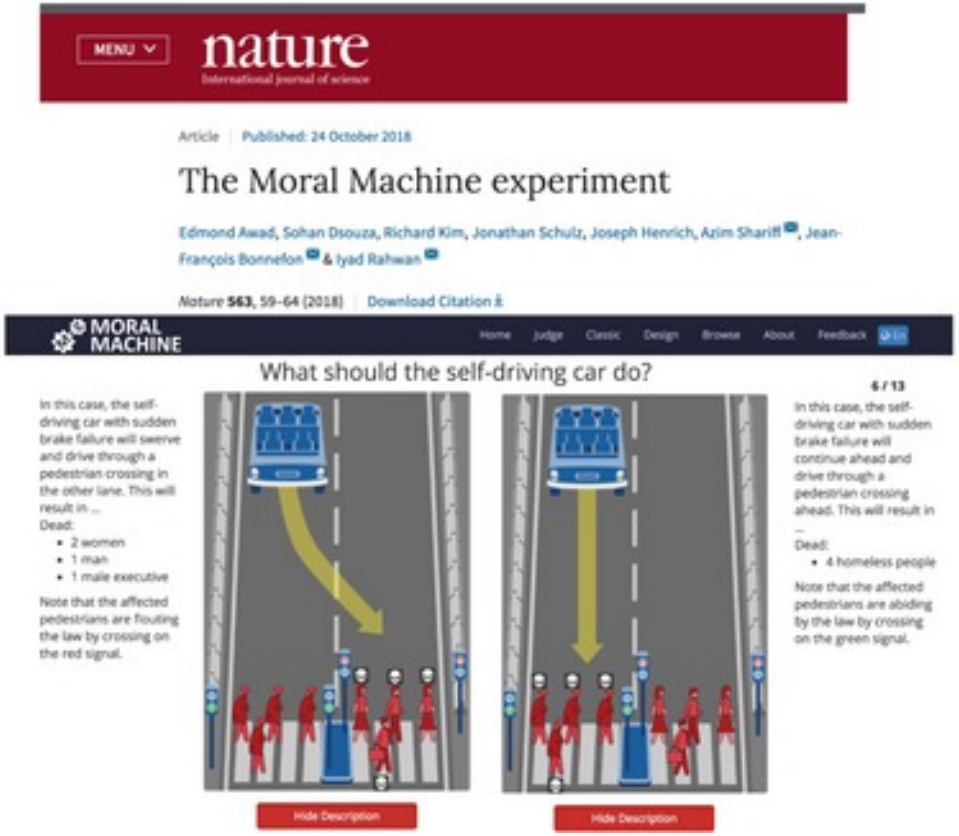
Bias in algorithms, in data, in use...



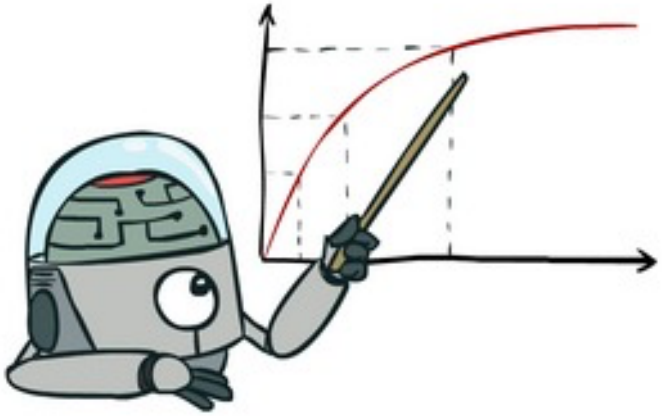
The algorithm was able to tell if a man is gay or straight using one picture 81pc of the time, and could determine a woman's sexuality 74pc of the time. Humans were much less accurate in comparison, correctly guessing just 61pc of the time for men and 54p for women.

Clearly raising numerous ethical issues ...

What would an ‘unbiased’ algorithm be?



Maximize Your Expected Utility

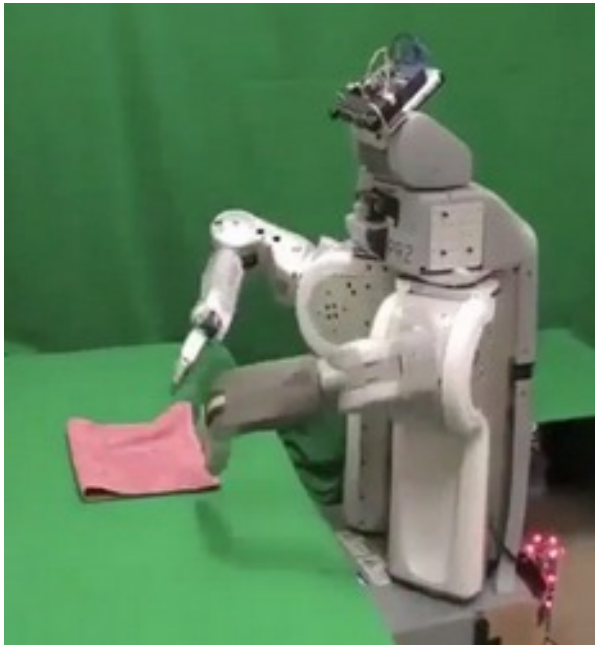


Utility?

Clear utility function



Not so clear utility function




SCORE
0
LAPS
–/3
TIME
0:03
TURBO
MORE GAMES

What expected utility is right?

In this course, everything will be unambiguous
toy-ish often even

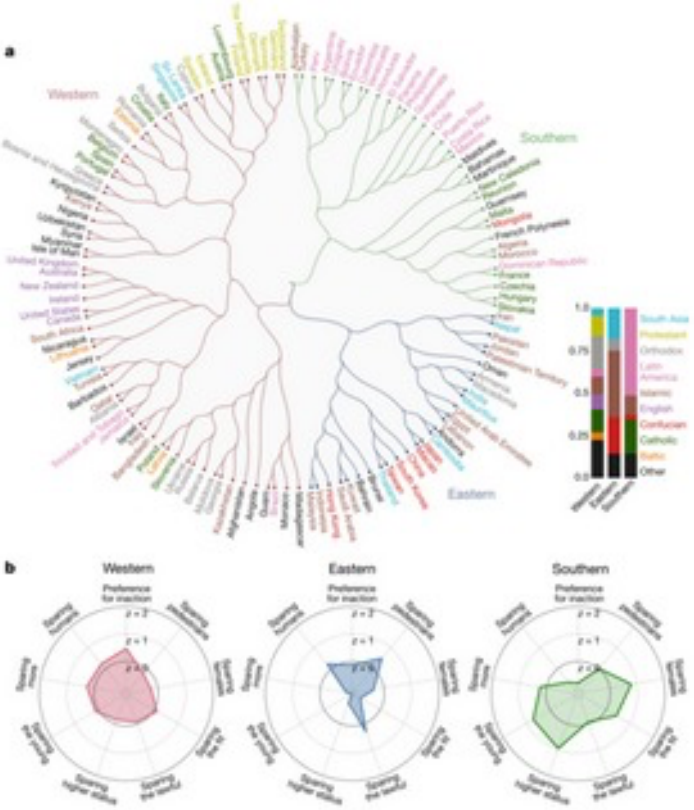
But the implications of AI in the world are real
Think about them, talk about them...

Guest lecture: Ethics in AI

On **Thu 17 December** we will have a Guest Lecture by Prof. Nathalie A. Smuha on Ethics of AI!

Nathalie A. Smuha is an Assistant Professor of Law at the University of Toronto Faculty of Law, where she examines legal, ethical and philosophical questions around Artificial Intelligence (AI) and other digital technologies. Her research focuses particularly on the impact of AI on human rights, democracy and the rule of law. She is the author of Algorithmic Rule By Law: How Algorithmic Regulation in the Public Sector Erodes the Rule of Law, and the editor of the Cambridge Handbook on the Law, Ethics and Policy of Artificial Intelligence (2025).

What would an ‘unbiased’ algorithm be?



Different societies have different norms!

So:

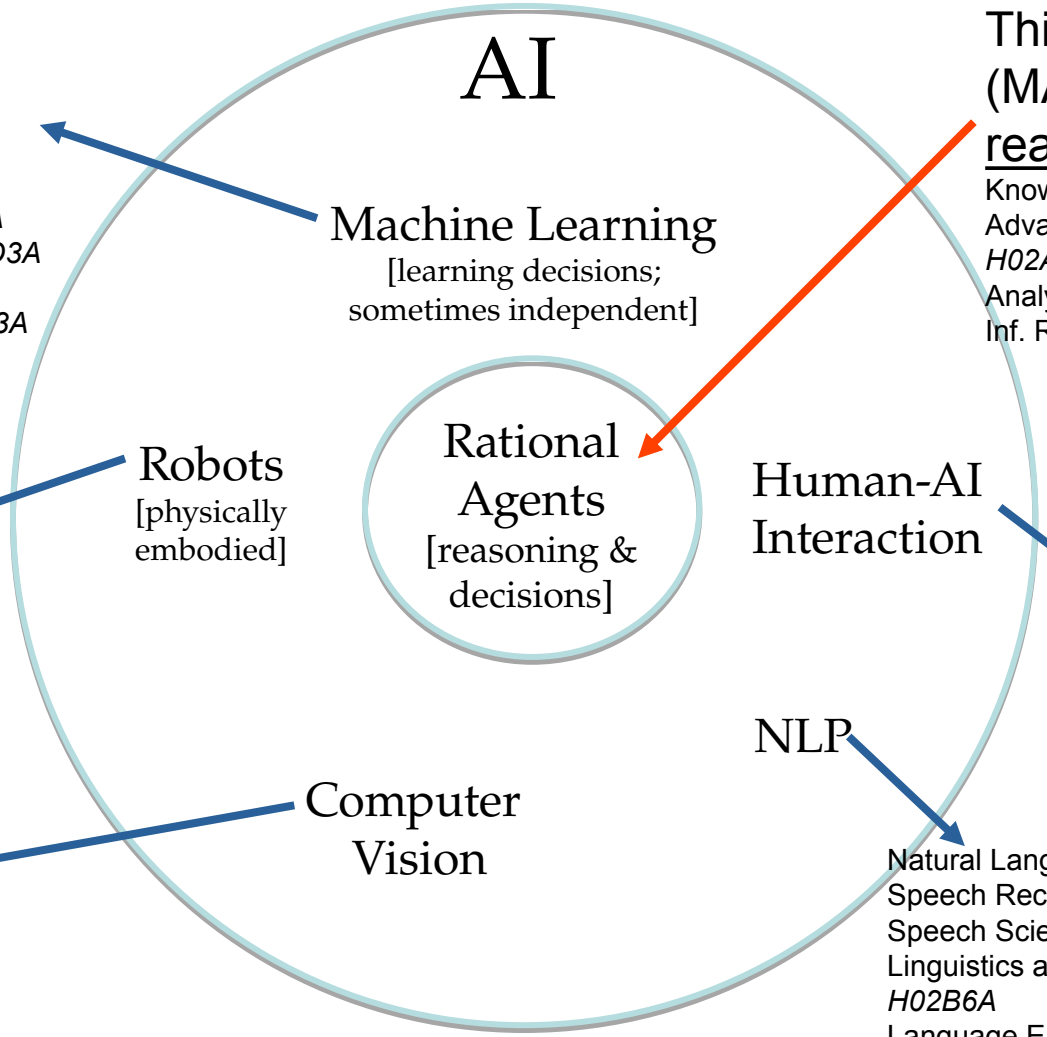
- AI Alignment is hard
- Determining what to align to is even harder

Goes back to *root* of what it means to be human…

What “expected utility” is right?

Further reading on implications of AI

- Website Stuart Russell, The Long Term Future of AI, also on Lethal Autonomous Weapons.
- Book Stuart Russell “Human Compatible” (2019) Nick Bostrom “Superintelligence” (2014)
- Moshe Vardi on jobs
- Slides from Asilomar panel on Long Term AI futures at IJCAI 2009;
- Research priorities for robust and beneficial artificial intelligence @ futureoflife.org
- AI and Life in 2030, https://ai100.stanford.edu/sites/default/files/ai_100_report_0831fnl.pdf
- EU High Level Expert Group https://ec.europa.eu/digital-single-market/en/news/ethics-guidelines-trustworthy-ai
- Preparing for the future of AI; https://obamawhitehouse.archives.gov/sites/default/files/whitehouse_files/microsites/ostp/NSTC/preparing_for_the_future_of_ai.pdf
- Beneficial ai — 23 Asilomar AI principles https://futureoflife.org/bai-2017/



Not in this course

Mach. Learning & Ind. Inf. *H02C1A*
(ML&II is MAI mandatory)
ANN & Deep Learning *H02C4A*
Support Vector Machines *H02D3A*
Data Mining *H02C6A*
Reinforcement Learning *H0O23A*

This course (MAI mandatory) reasoning by search

Knowledge Representation *H02C3A*
Advanced Prog. Lang. for AI *H02A8A*
Analysis of LS Social Netw. *H02T6A*
Inf. Retrieval & Search Eng. *H02C8A*

Robotics *H02A4A*

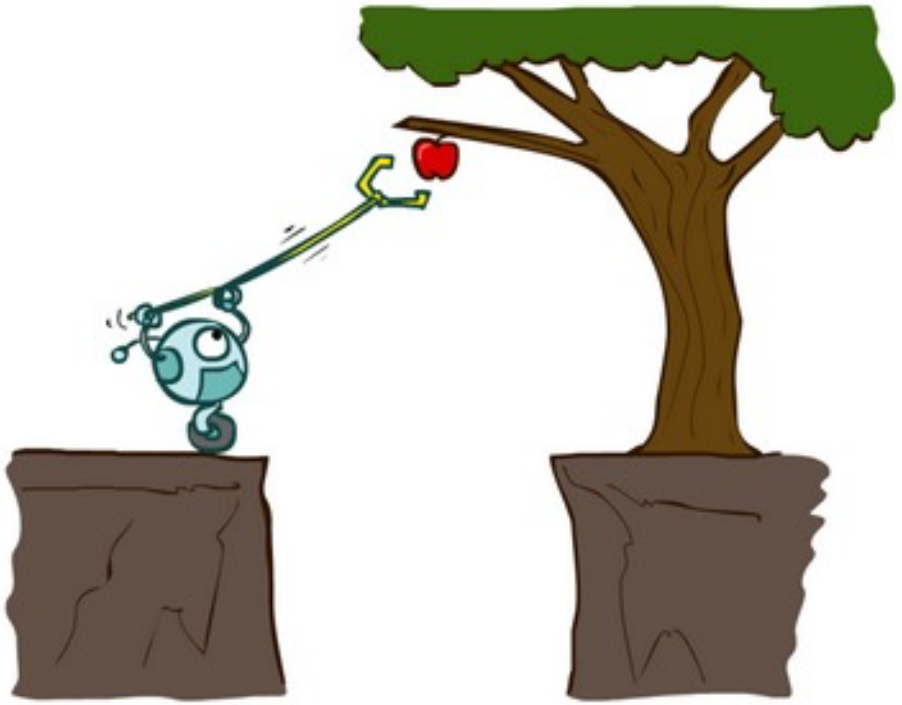
Cognitive Science *H02B2A*
Privacy and Big Data *H00Y2A*
Philosophy of Mind and Artificial Intelligence *H02D5A*
AI Ethics & Regulation *H0P05A*

Computer Vision *H02A5A*

Natural Language Processing *H02B1A*
Speech Recognition *H02A6A*
Speech Science *H02C9A*
Linguistics and Artificial Intelligence *H02B6A*
Language Engineering Applications

KU LEUVEN

Fundamentals of Artificial Intelligence



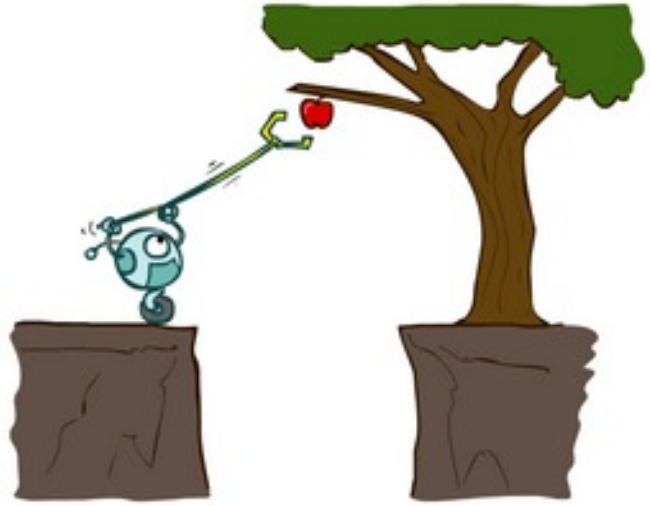
Prof. Tias Guns

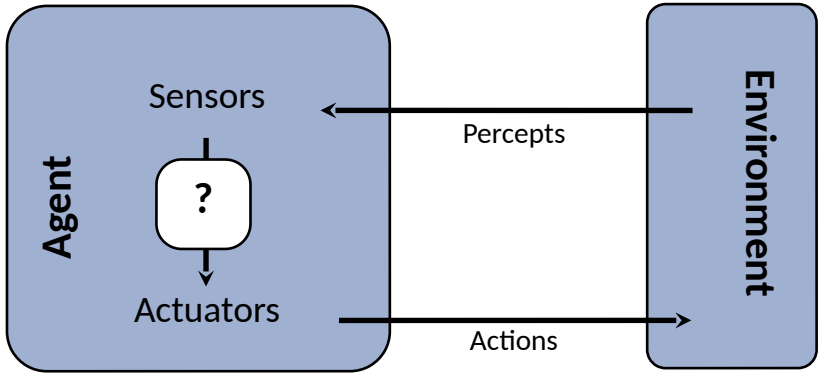
Lecture 02, 2026-27
Agents

Based on the slides of CS188 @ Berkeley

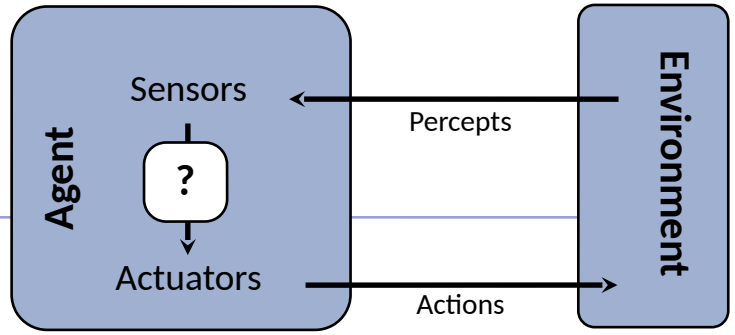
RECAP: Designing Rational Agents

- An **agent** is an entity that *perceives* and *acts*.
- A **rational agent** selects actions that maximize its (expected) **utility**.
- Characteristics of the **percepts, environment,** and **action space** dictate techniques for selecting rational actions
- **This course** is about:
  - General AI **search** techniques for a variety of problem types
  - Learning to recognize when and how a new problem can be solved with an existing technique





Environment types (1/2)



The environment type largely determines the agent design

Fully or Partially observable

- Are the relevant aspects of the environment accessible to the sensors?
- ***Partially observable*** => agent requires ***memory*** (internal state)

Deterministic vs. stochastic

- Is the next state of the environment completely determined by the current state and the selected action?
- ***Stochastic*** => actions may *fail* or environment may respond nondeterministically, agent has to prepare for ***contingencies***
- If only actions of other agents are nondeterministic, the environment is called *strategic deterministic*.

Static vs. dynamic

- Can the environment change *while the agent is deliberating*?
  (If the environment does not change but if the agent's performance score changes as time passes by the environment is denoted as semi-dynamic.)
- ***Static*** => agent has time to compute a rational decision

Environment types






	Pacman	Backgammon	Taxi	MS CoPilot Chat
Fully or partially observable				
Deterministic or stochastic				
Static or dynamic				

suggest something healthy for me to eat this evening

A few healthy options, depending on your mood:

- **Salmon + roasted vegetables + quinoa**
  - High protein, healthy fats, filling without being heavy.
  - Roast broccoli, carrots, and zucchini in the oven.
- **Chicken stir-fry**
  - Chicken, lots of mixed vegetables, ginger, garlic, soy sauce.
  - Serve with brown rice.
- **Mediterranean bowl**
  - Chickpeas, cucumber, tomato, feta, olives, mixed greens.
  - Add grilled chicken or halloumi if you're hungrier.
- **Vegetable omelette**
  - Eggs, spinach, mushrooms, onion, tomato.
  - Small side salad and whole-grain bread.
- **Lentil curry**
  - Lentils, tomatoes, spinach, onion, spices.
  - Healthy, cheap, and keeps well.

If you've had a mentally demanding day and want something satisfying **salmon, roasted vegetables, and quinoa**. It's hard to go wrong with th

suggest something healthy for me to eat this evening

Given your profile (researcher, likely not looking to spend an hour cooking), I'd go for one of these:

1. **Salmon + roasted vegetables + quinoa**
   - Salmon fillet in the oven (15 min)
   - Broccoli, carrots, courgette roasted alongside
   - Small portion of quinoa
   - High protein, healthy fats, lots of vegetables
2. **Mediterranean bowl**
   - Chickpeas
   - Tomato, cucumber, red onion
   - Feta
   - Olives
   - Whole-grain pita
   - Olive oil + lemon dressing
3. **Thai-style stir fry**
   - Chicken or tofu
   - Mixed vegetables
   - Ginger, garlic, soy sauce
   - Served with brown rice
   - Fast, filling, and reasonably healthy
4. **Belgian autumn option**
   - Vegetable soup (pumpkin, leek, or tom
   - Whole-grain bread

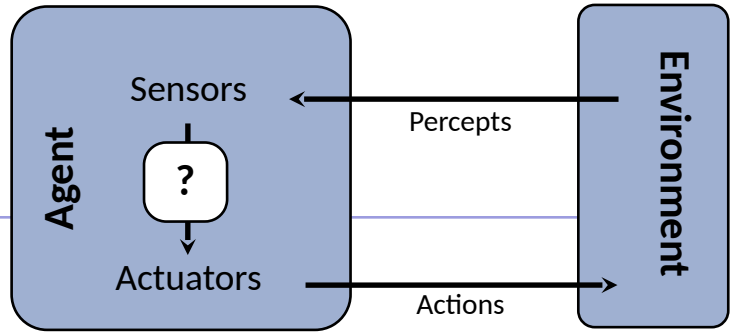
suggest something healthy for me to eat this evening

A few healthy options, depending on your mood:

- **Salmon, roasted vegetables, and quinoa**: high in protein, healthy fats, and keeps you full without feeling heavy.
- **Chicken stir-fry** with lots of vegetables (broccoli, peppers, mushrooms) and a moderate portion of rice.
- **Lentil curry** with spinach and vegetables. Very filling, lots of fiber and protein.
- **Large salad bowl** with mixed greens, chickpeas, avocado, cherry tomatoes, cucumber, feta, and some whole-grain bread.
- **Omelette** with vegetables, a side salad, and a slice of whole-grain toast. Quick and nutritious.
- **Mediterranean bowl**: hummus, grilled vegetables, couscous or bulgur, olives, and grilled chicken or falafel.

If you've had a sedentary day behind the computer, I'd personally go for **salmon + vegetables + quinoa** or **lentil curry**: lots of nutrients, good satiety, and not too many ultra-processed ingredients.

Environment types (2/2)



The environment type largely determines the agent design

Discrete vs. continuous

- Is the environment discrete (chess) or continuous (a robot moving in a room)?
- ***Continuous time*** => continuously operating ***controller***

Single agent vs. multi-agent

- Which entities have to be regarded as agents? There are competitive and cooperative scenarios.
- ***Multi-agent*** => agent may need to behave ***randomly***

Known environment?

- ***Unknown physics*** => need for ***exploration***

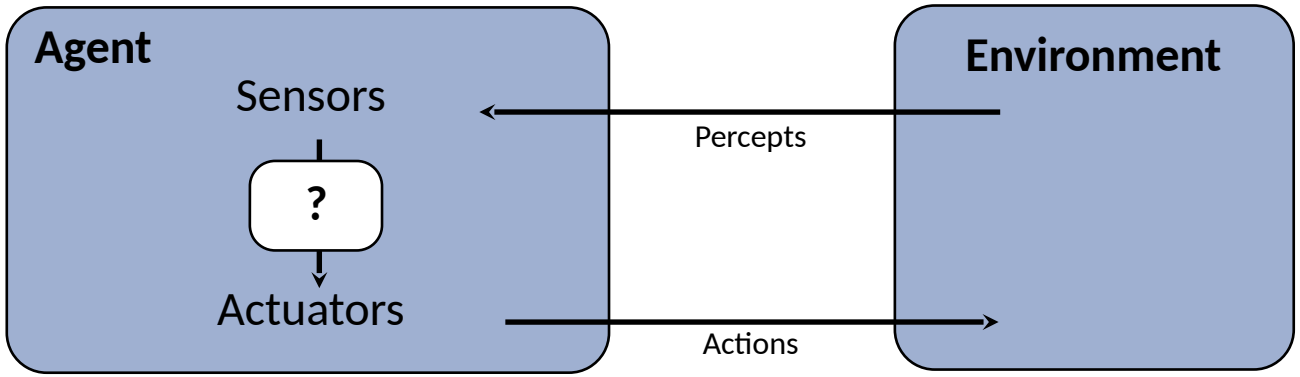
Known performance measure/evaluation function?

- ***Unknown perf. measure*** => observe/interact with ***human principal***

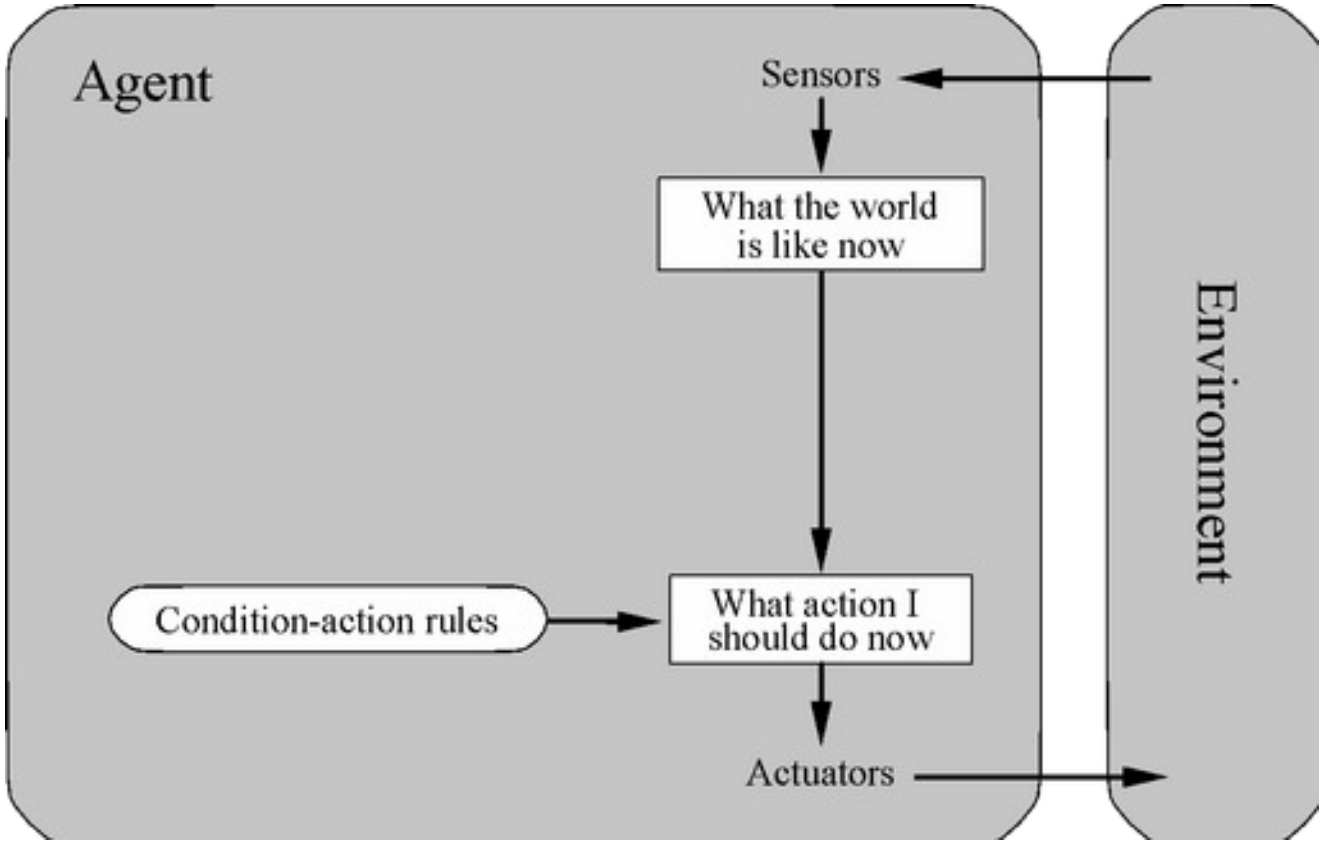
Environment types

	Pacman	Backgammon	Taxi	MS CoPilot Chat
Fully or partially observable				
Deterministic or stochastic				
Static or dynamic				
Discrete or continuous				
Single-agent or multi-agent				
Known physics?				
Known perf. measure?				

Example agent designs



Simple reflex agents

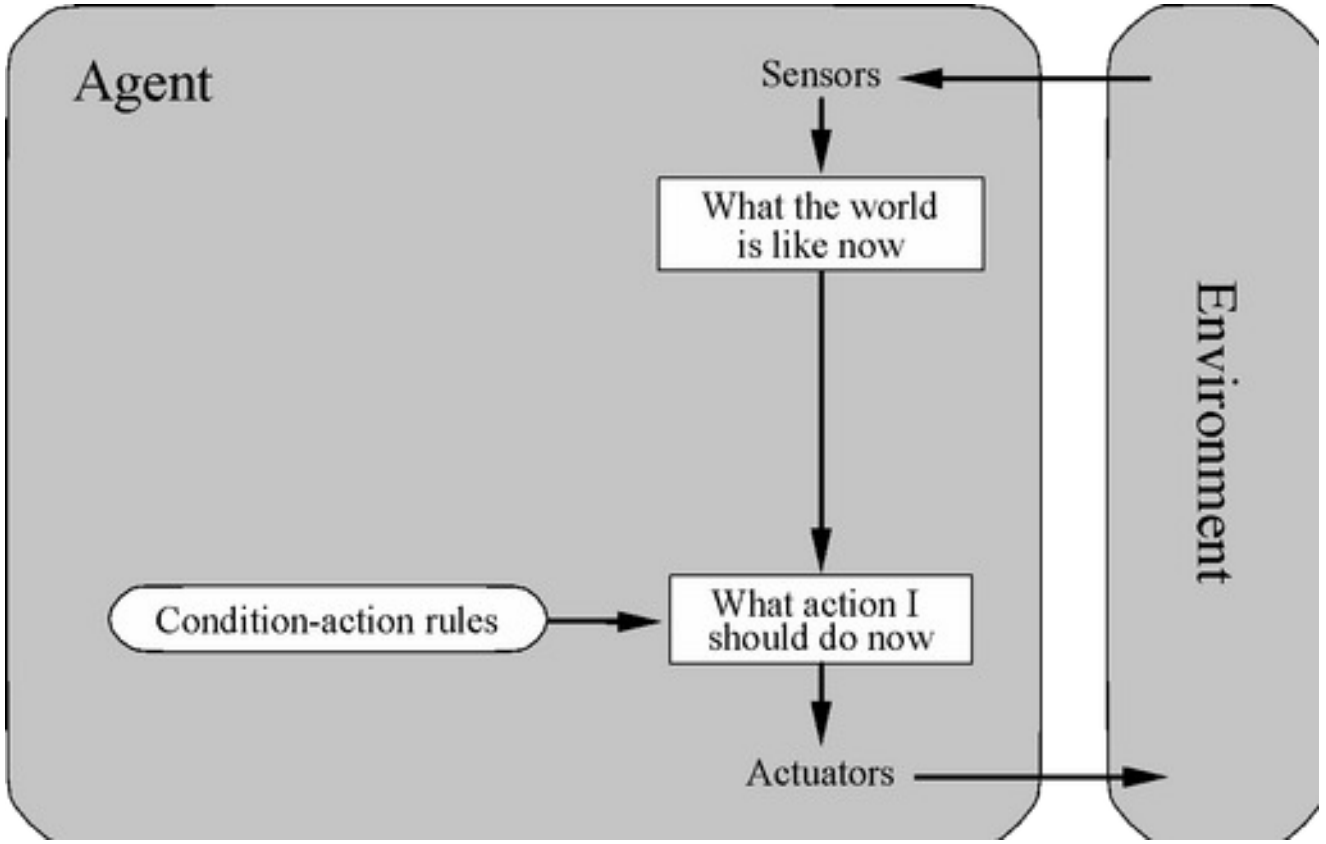


Pacman *agent program* in Python

```
class GoWestAgent(Agent):

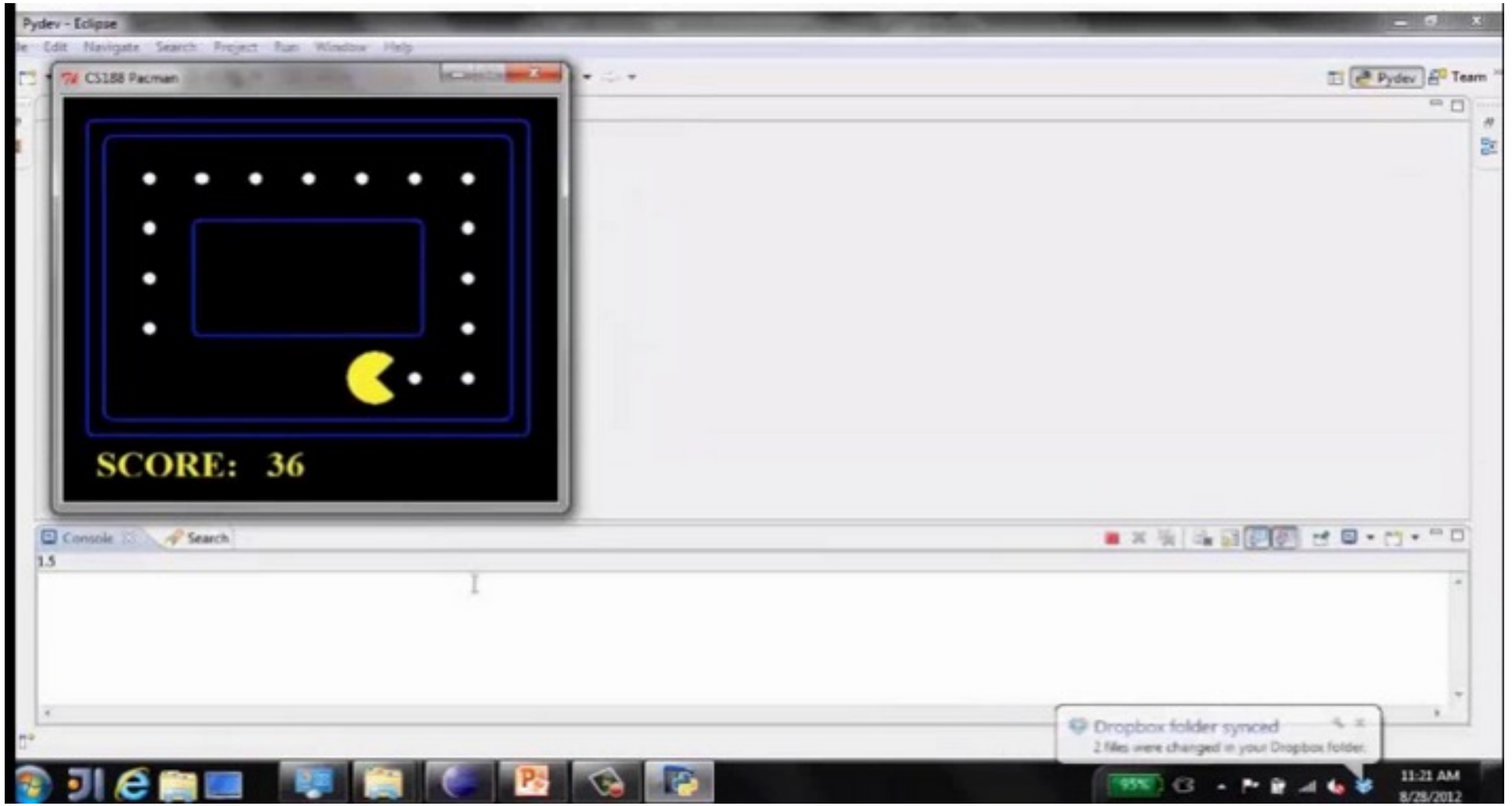

    def getAction(self, percept):
        if Directions.WEST in percept.getLegalPacmanActions():
            return Directions.WEST
        else:
            return Directions.STOP
```

Simple reflex agents

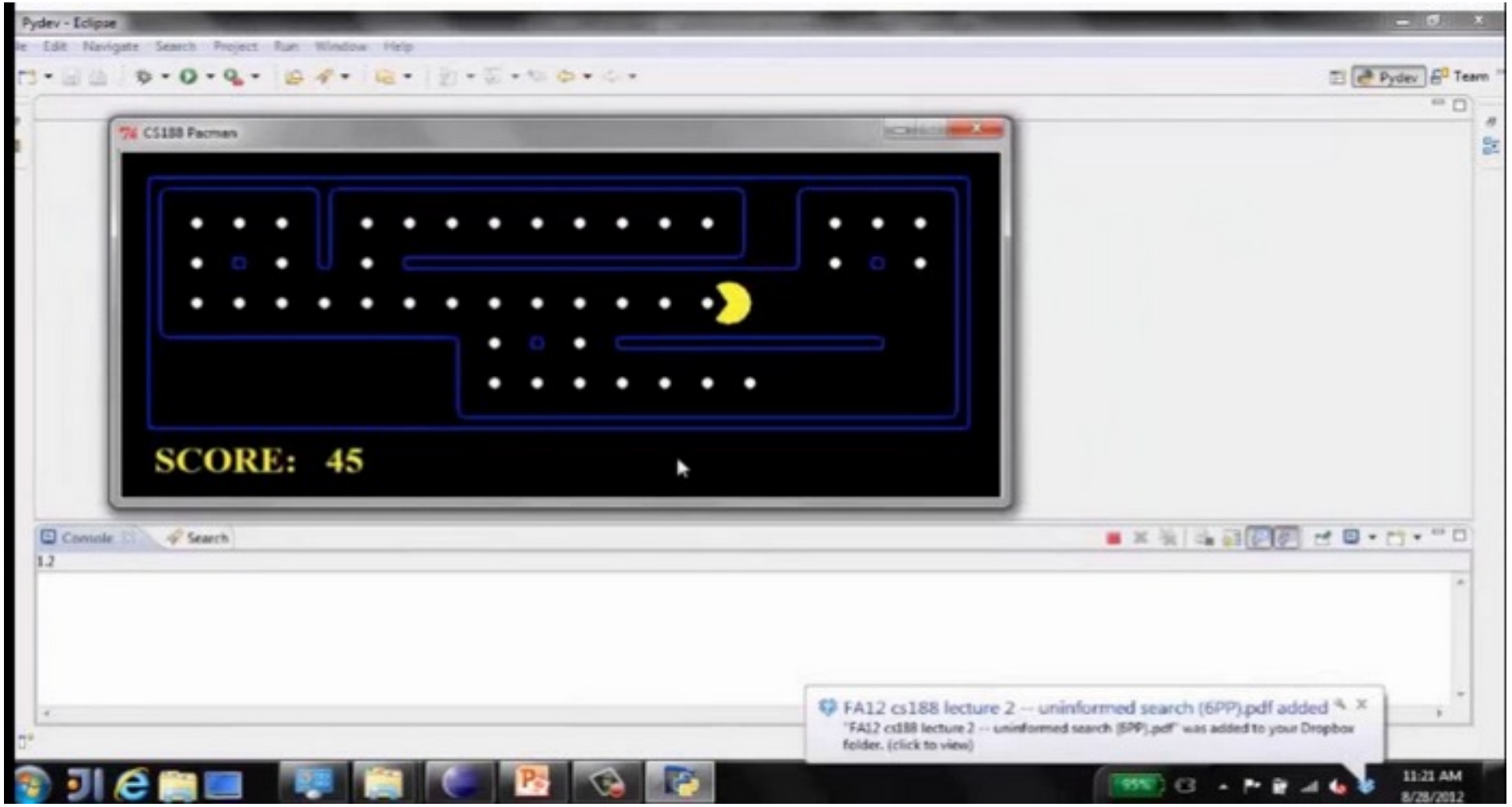


- Does not keep track of past. Only possible if environment is *fully observable*.
- Simple design, but the rules need to cover all cases...

Eat adjacent dot, if any



Eat adjacent dot, if any

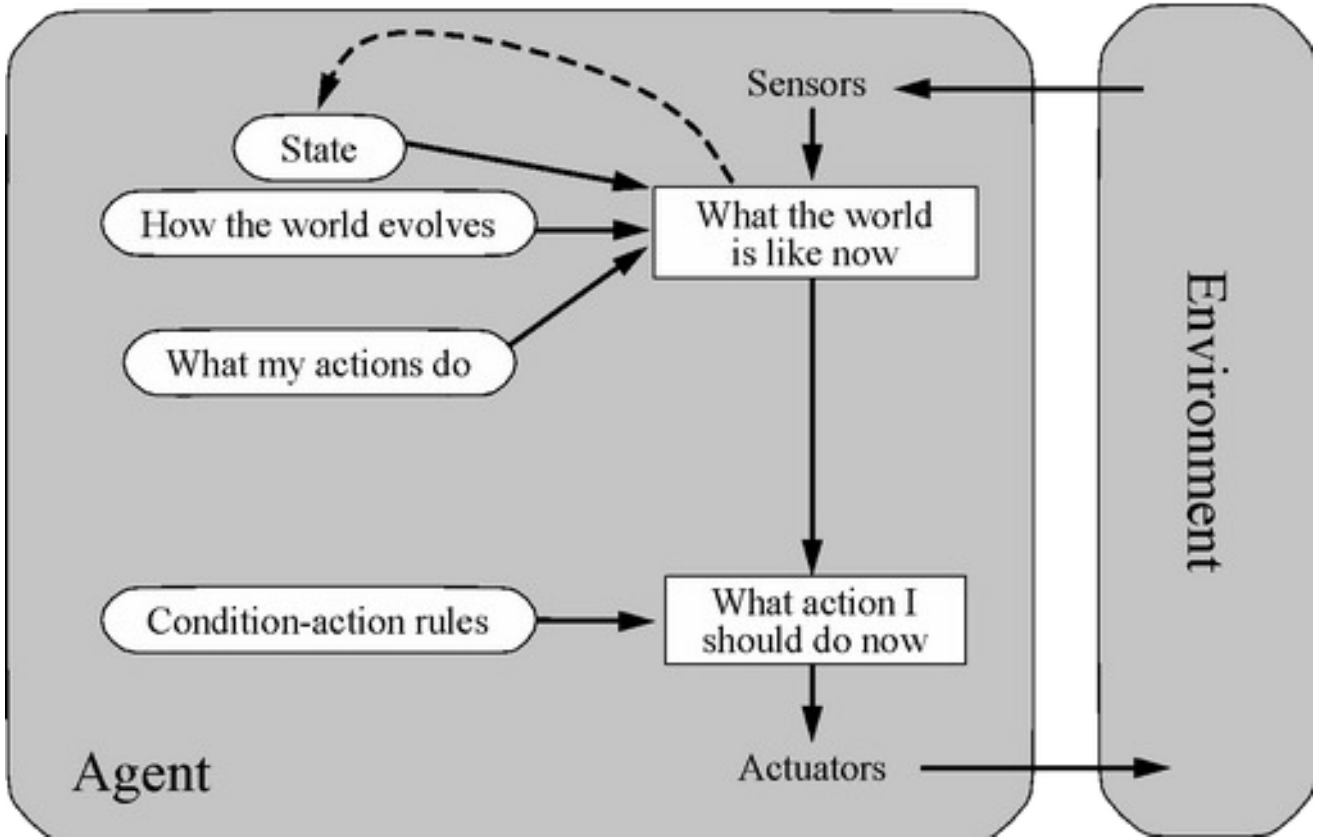


Pacman agent contd.

- Can we (in principle) extend this reflex agent to behave well in all standard Pacman environments?
  - No – Pacman is not quite fully observable (power pellet duration)
  - Otherwise, yes – we can (*in principle*) make a lookup table…..
  - *How large would it be?*

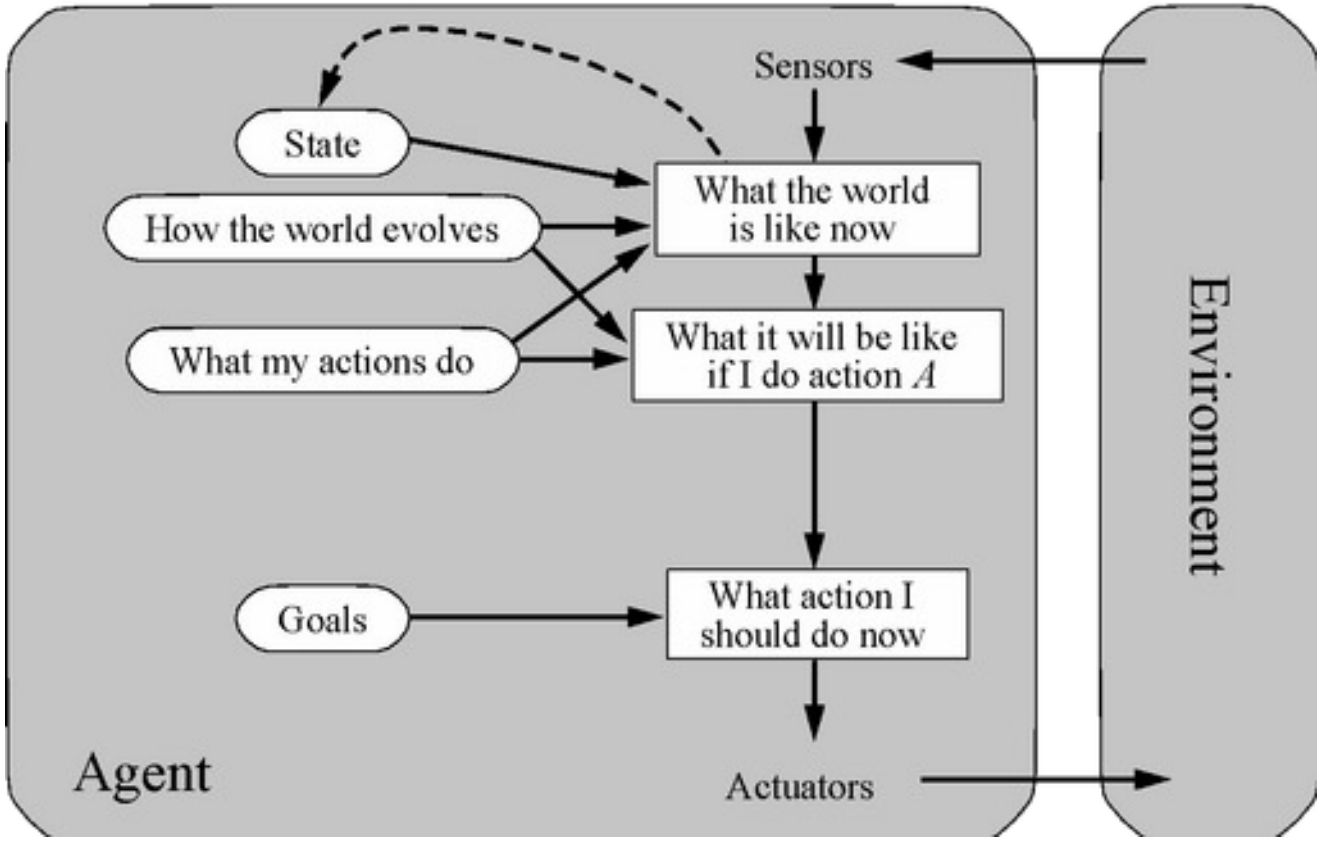
- Can every function be implemented by some agent program?
  - No – Consider agent for chess on a slow computer, NP-hard problems, halting problem

Model-based agents



- Keeping track of past? Store internal state, update it every time a percept is received
- Needs transition model (what my actions do/did) and sensor model (how the world evolves)

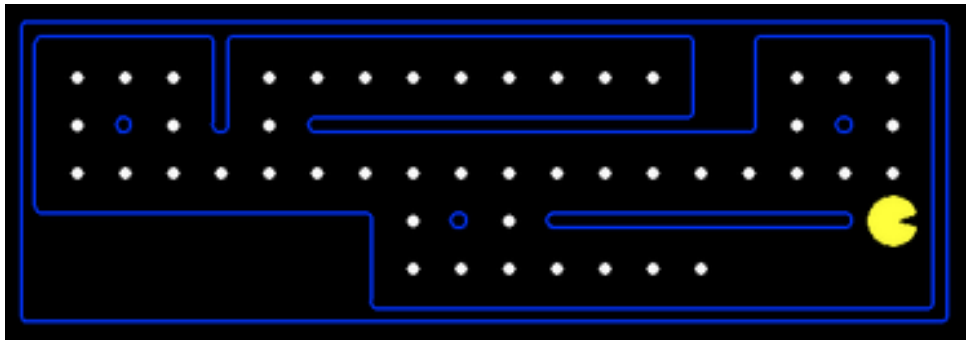
Goal-based agents

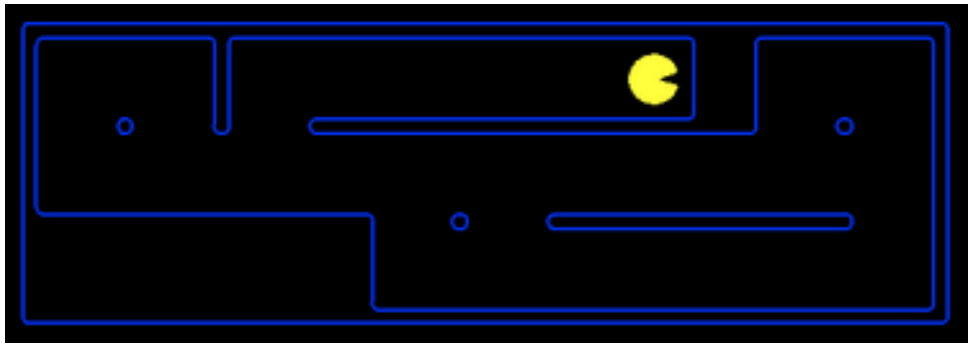


- Reasons about reaching a (multi-step) goal, using its models.
- (most of the agents we will study in this course)

Goal-based Agents

- Goal-based agents (also called planning agents) decide based on evaluating future action sequences
- Must have a model of how the world evolves in response to actions
- Have a definite goal (or multiple ones)
- Optimal: Achieve goal at lowest possible cost



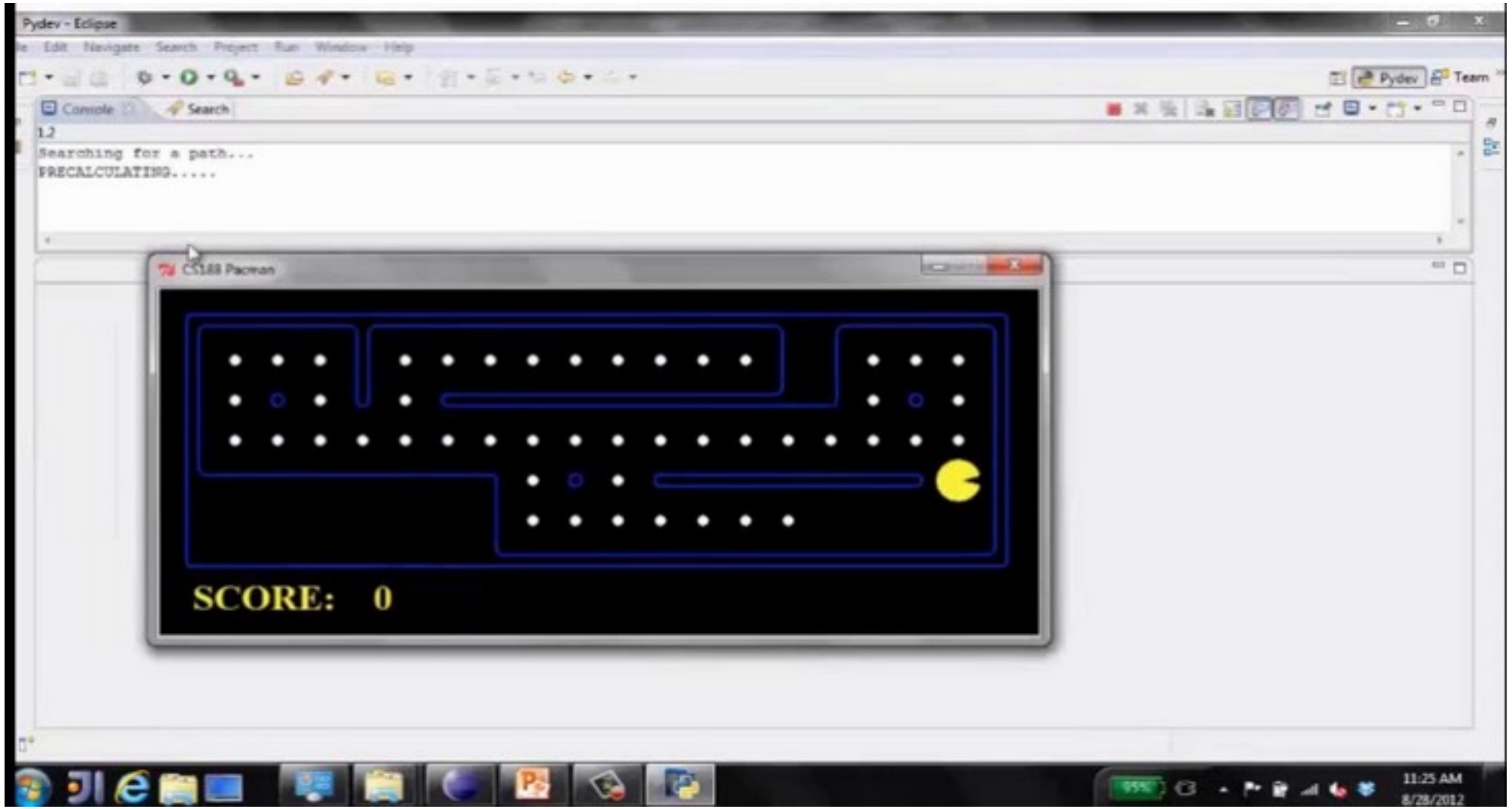


Summary



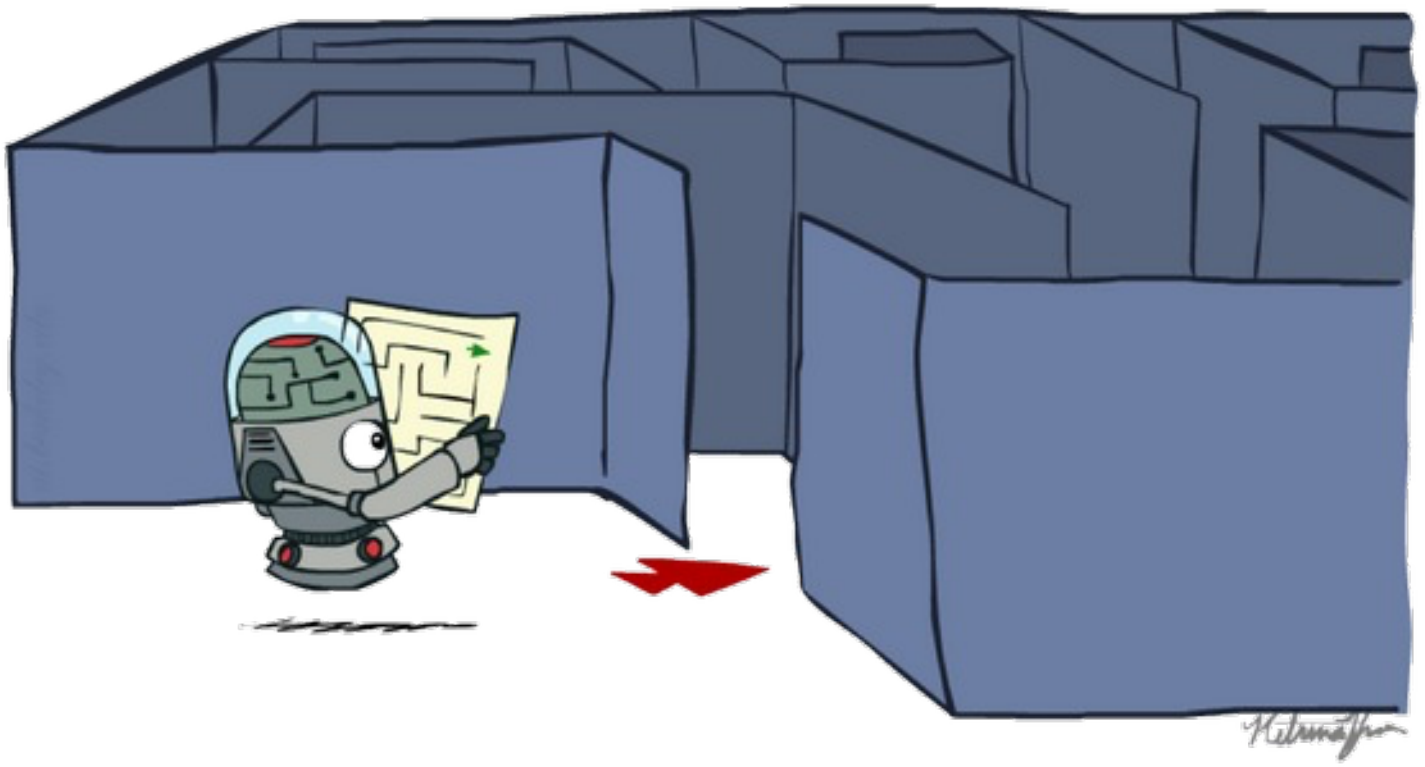
- An agent is something that perceives and acts. It consists of an architecture and an agent program.
- A rational agent takes the action that maximizes its expected utility given the percept sequence and its knowledge of the environment.
- An agent program maps from a percept to an action. There are a variety of designs
  - Reflex agents respond immediately to percepts.
  - Goal-based agents work towards goals.
  - Not discussed: learning agents improve their behaviour over time.
- Some environments are more demanding than others.
- Environments that are partially observable, nondeterministic, dynamic, and multi-agent are the most challenging.

Precompute optimal plan, execute it



KU LEUVEN

Fundamentals of Artificial Intelligence



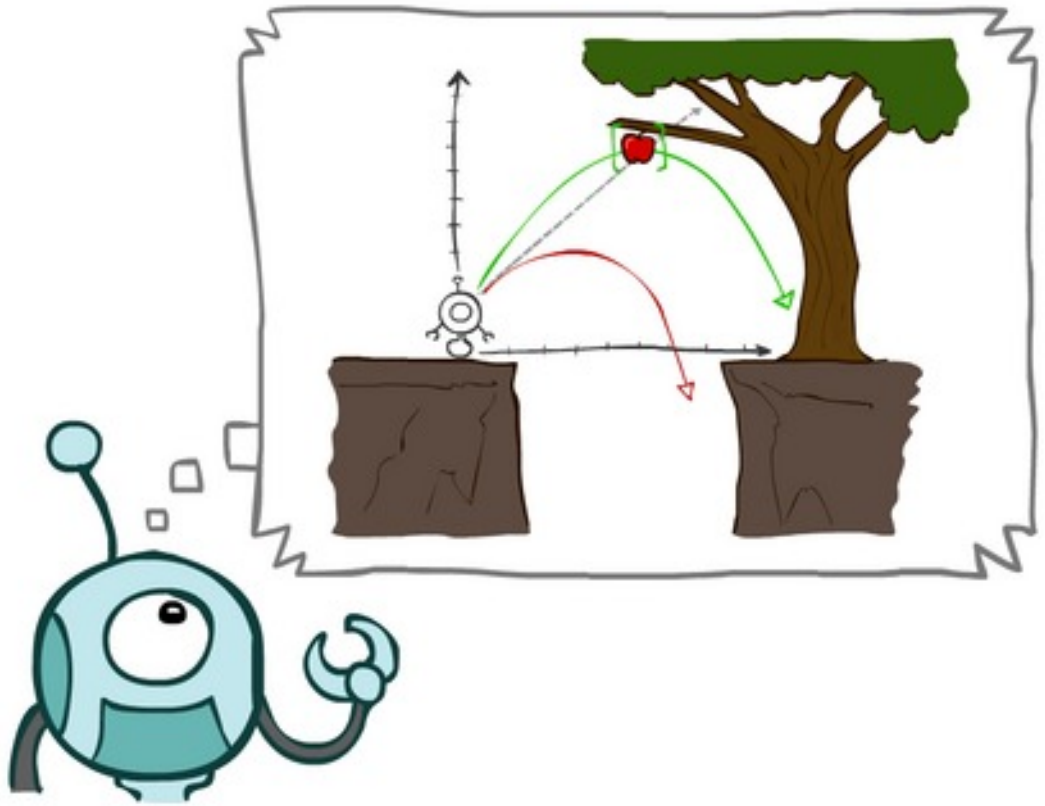
Prof. Tias Guns

Lecture 03, 2026-27
Search

Based on the slides of CS188 @ Berkeley

Content

- Search Problems
- Uninformed Search Methods
  - Depth-First Search
  - Breadth-First Search
  - Uniform-Cost Search



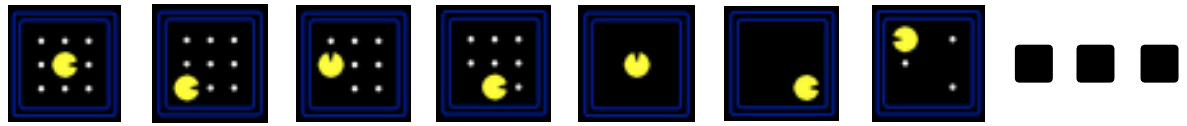
At the end of this chapter, you are ready to solve project P1's Q1 and Q2!

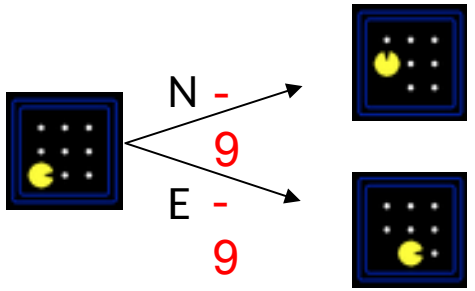
Search Problems



Search Problems

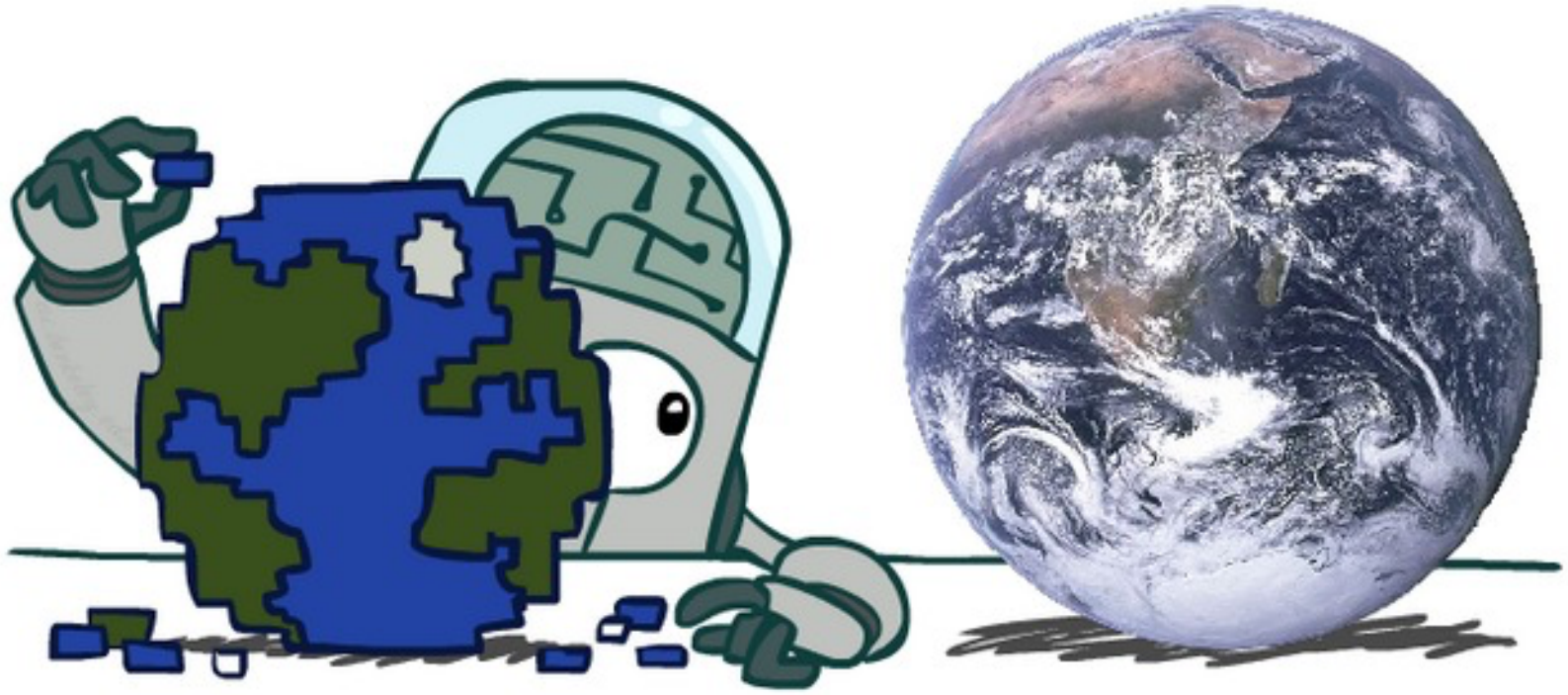
- A search problem consists of:
  - A state space S
  - An initial state s_0
  - Actions $A(s)$ in each state
  - Transition model $Result(s,a)$
  - A goal test $G(s)$
    - s has no dots left
  - Action cost $c(s,a,s')$
    - +1 per step; -10 food; -500 win; +500 die; -200 eat ghost





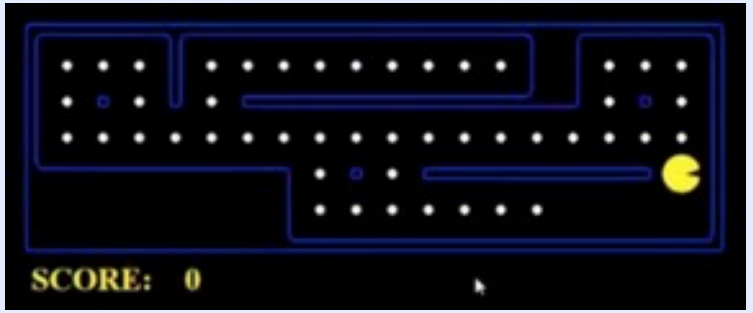
- A solution is an action sequence that reaches a goal state
- An optimal solution is a solution with minimal cost

Search Problems Are Models



What's in a State Space?

The world state includes every last detail of the environment



A search state keeps only the details needed for planning (abstraction)

- Problem: Pathing
  - States: (x,y) location
  - Actions: NSEW
  - Transition: update x,y value
  - Goal test: is (x,y)=destination

- Problem: Eat-All-Dots
  - States: {(x,y), dot Booleans}
  - Actions: NSEW
  - Transition: update x,y and possibly a dot Boolean
  - Goal test: dots all false

State Space Sizes

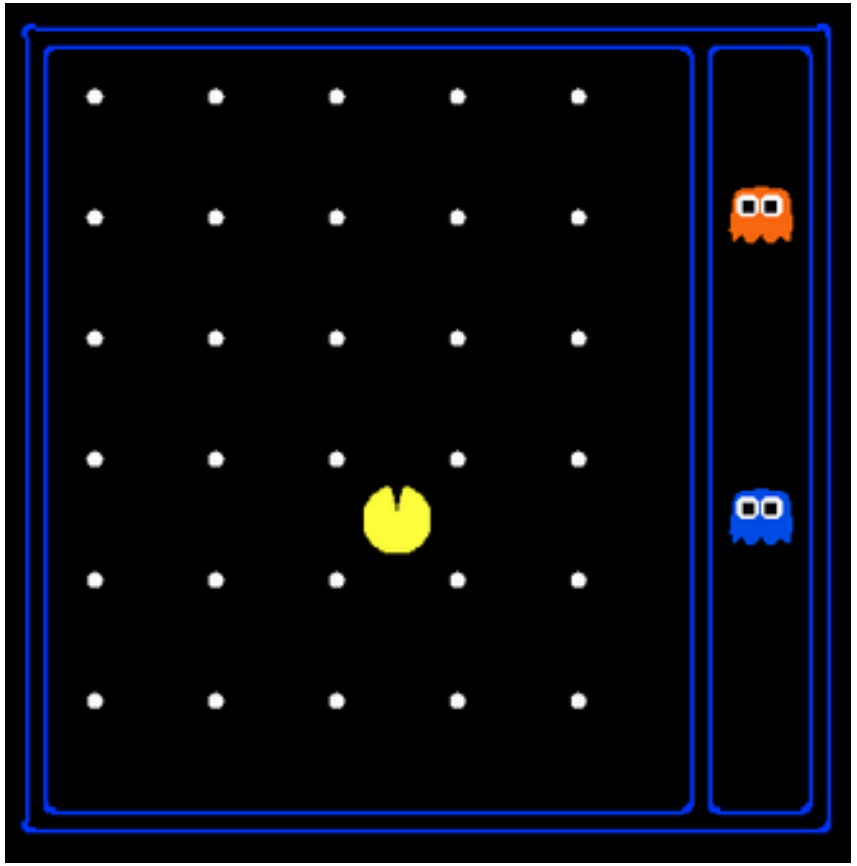
- World state:
  - Agent positions: 120
  - Food count: 30
  - Ghost positions: 12
  - Agent facing: NSEW

- How many
  - World states?

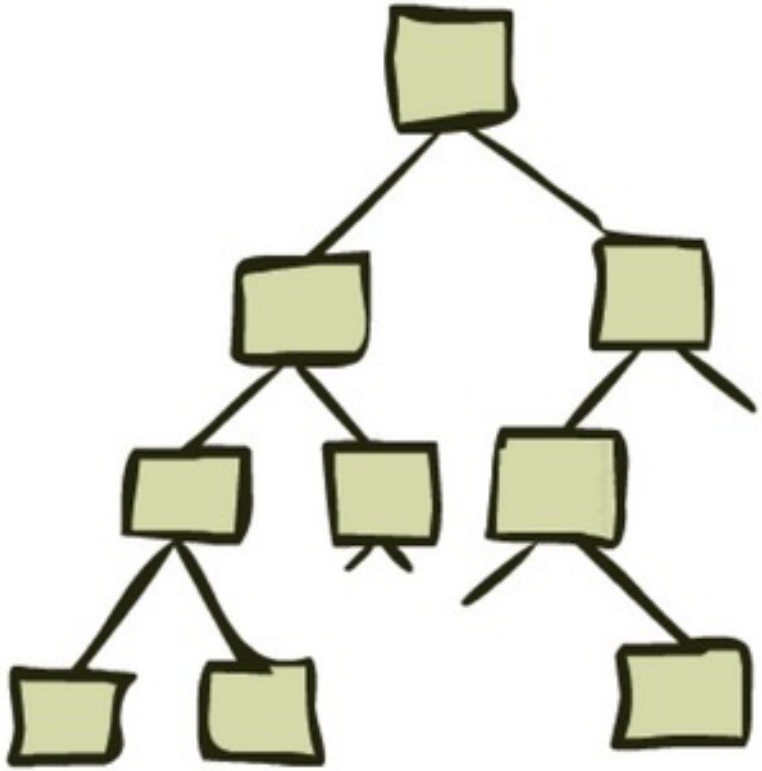
    $120x(2^{30})x(12^2)x4$
  - States for pathing?

    120
  - States for eat-all-dots?

    $120x(2^{30})$

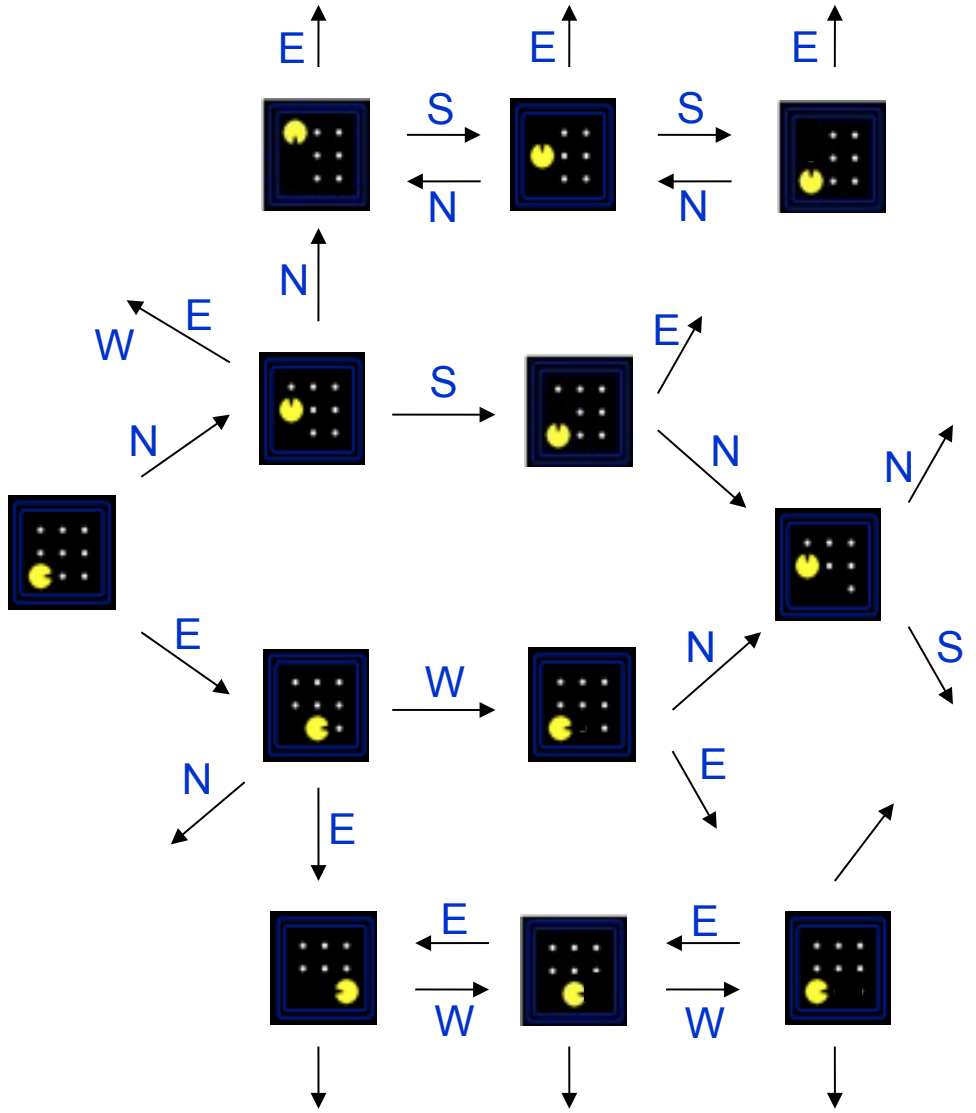


State Space Graphs and Search Trees



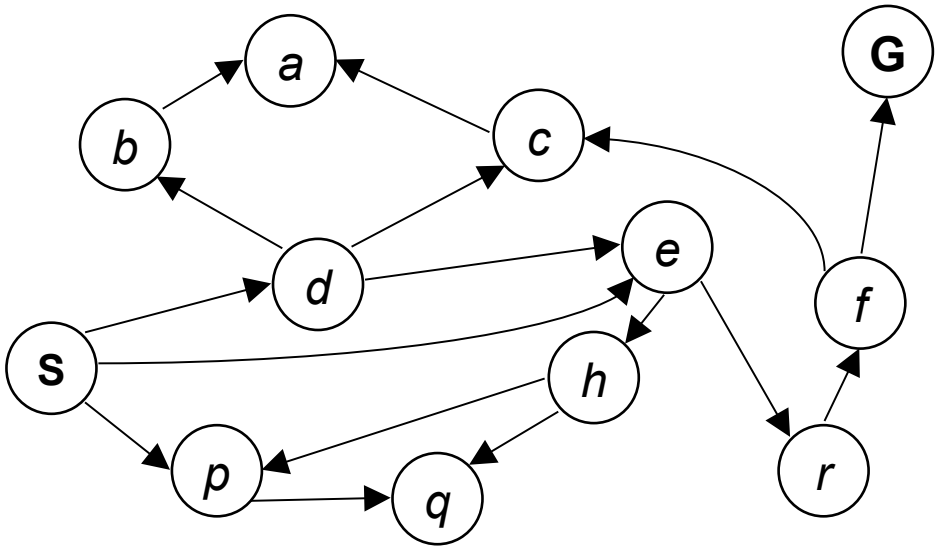
State Space Graphs

- State space graph: A mathematical representation of a search problem
  - Vertices are (abstracted) world configurations
  - Edges represent transitions (labeled with actions)
  - The goal test is a set of goal vertices (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



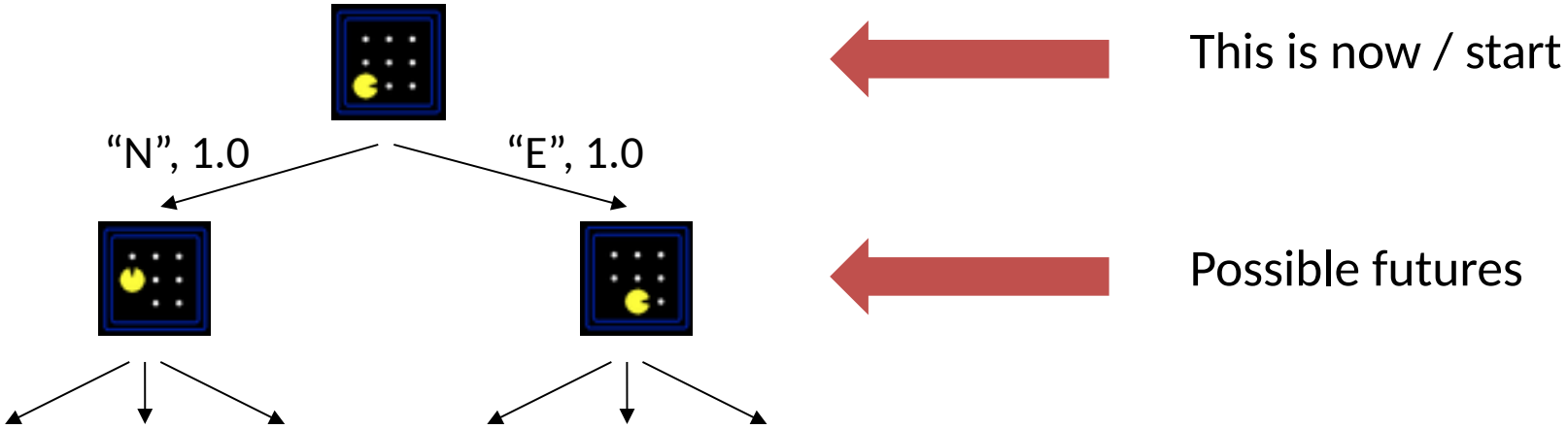
State Space Graphs

- State space graph: A mathematical representation of a search problem
  - Vertices are (abstracted) world configurations
  - Edges represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



Tiny state space graph for a tiny search problem

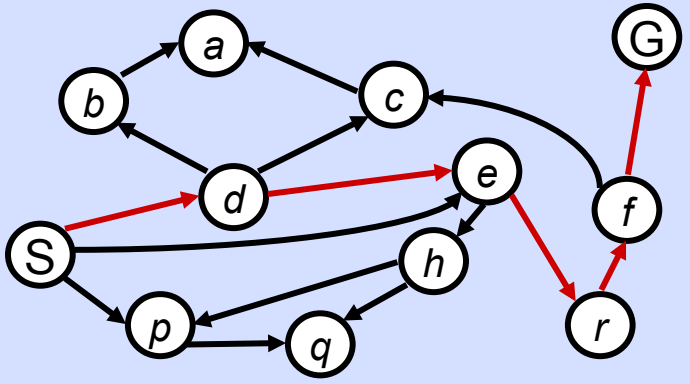
Search Trees



- A search tree:
  - A "what if" tree of plans and their outcomes
  - The start state is the root node
  - Children correspond to successors
  - Nodes show states, but correspond to PLANS (=sequence of states) that achieve those states
  - For most problems, we can never actually build the whole tree

Search Trees through the State Space Graphs

State Space Graph

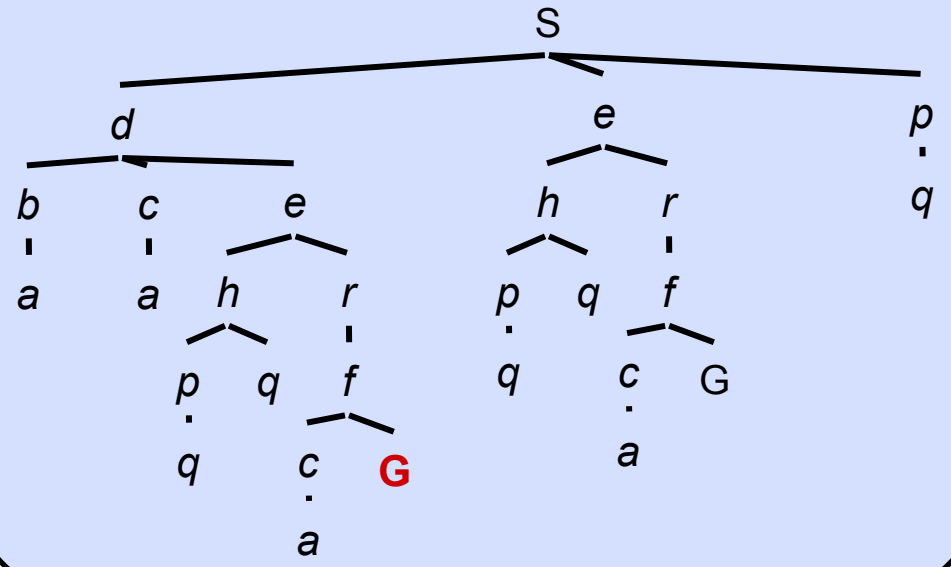


We construct the tree on demand – and we construct as little as possible.

Pay attention:

A search tree NODE (the red G →) will represent an entire PATH in the state space graph (← the red path)

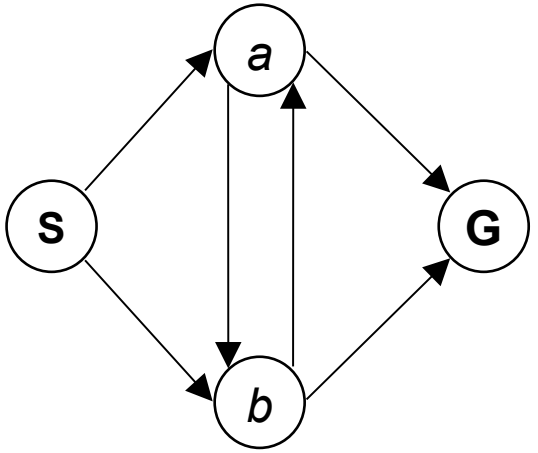
Search Tree



On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

Consider this 4-state graph:

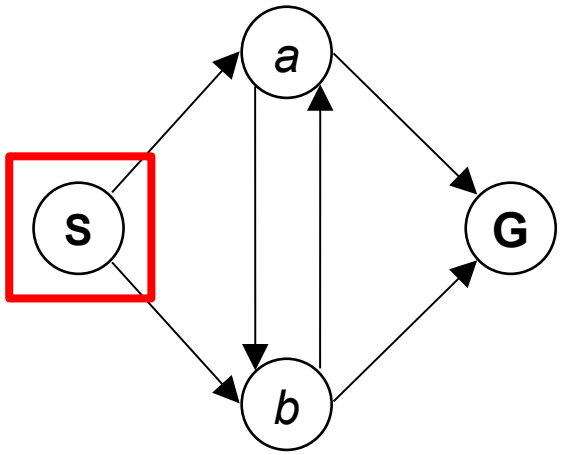


On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S> *(sequence with one element: S)*

Consider this 4-state graph:



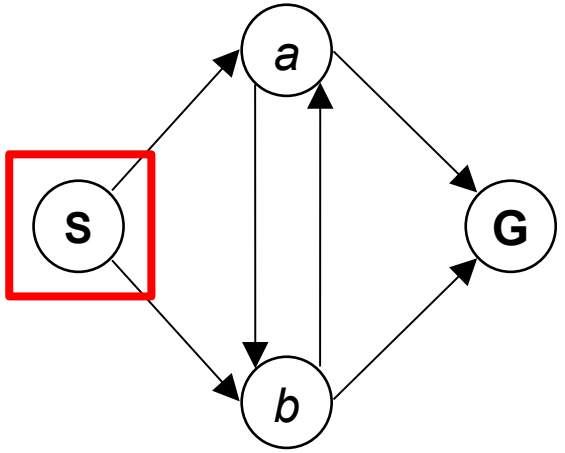
On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>

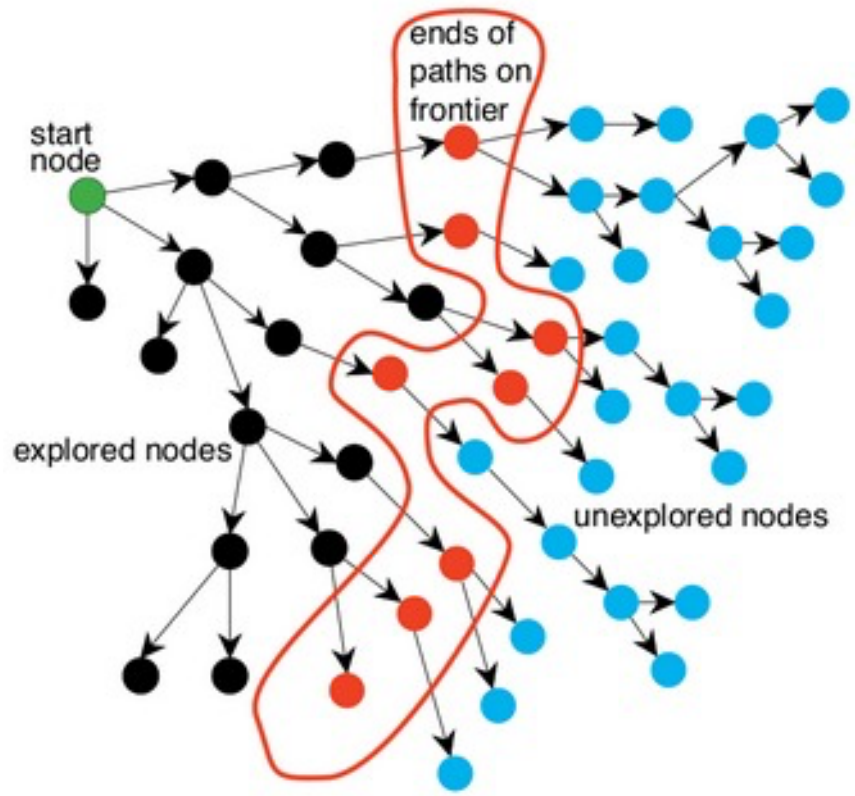
frontier = {<S>} (set with one element: the sequence <S>)

Consider this 4-state graph:



The expanding frontier

- Node in search tree is path in state-space graph
- frontier / fringe: set of all nodes available for expansion
- node expansion: generating all successor nodes considering the available actions

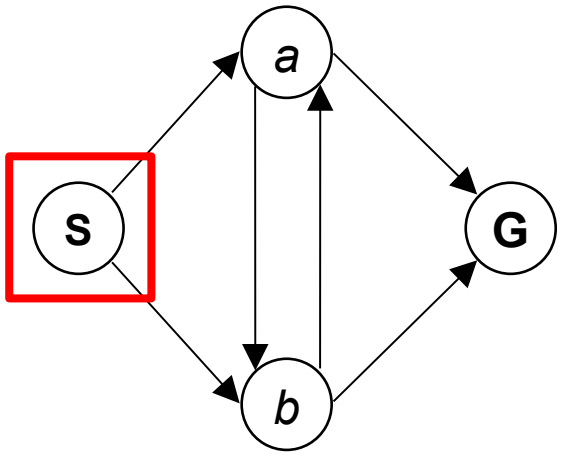


On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>
frontier = {<S>}

Consider this 4-state graph:

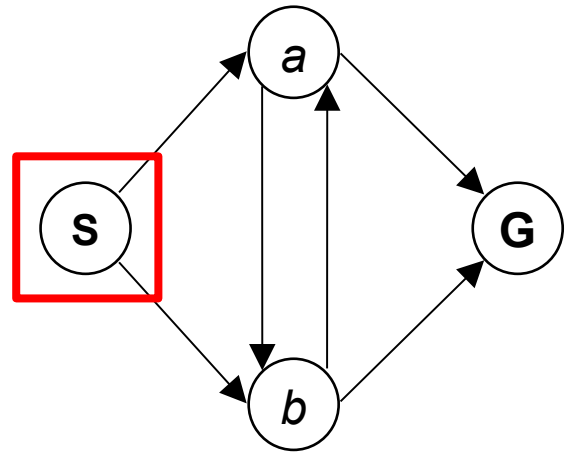


On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>
frontier = {}
node = <S>

Consider this 4-state graph:

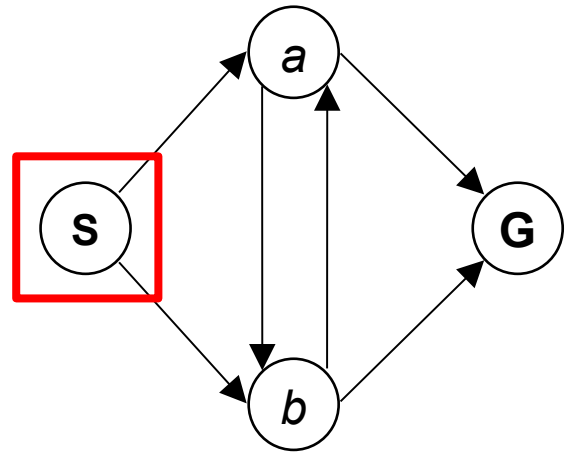


On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>
frontier = {}
node = <S>
node.STATE = S

Consider this 4-state graph:

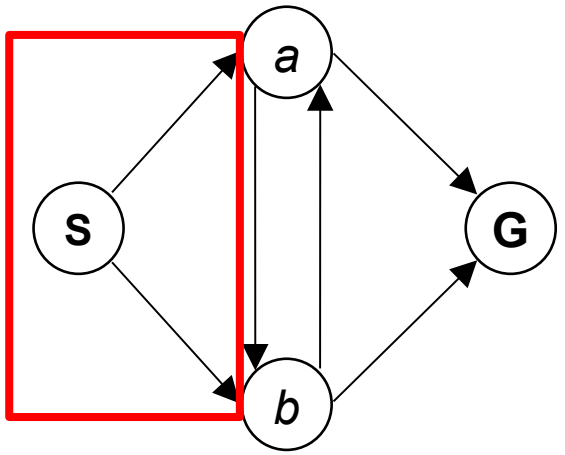


On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>
frontier = {}
node = <S>
node.STATE = S
child-state = a

Consider this 4-state graph:

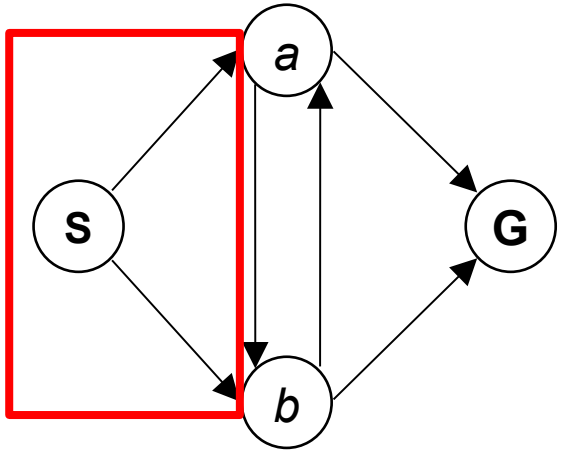


On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>
frontier = {}
node = <S>
node.STATE = S
child-state = a
child-node = <S,a>

Consider this 4-state graph:

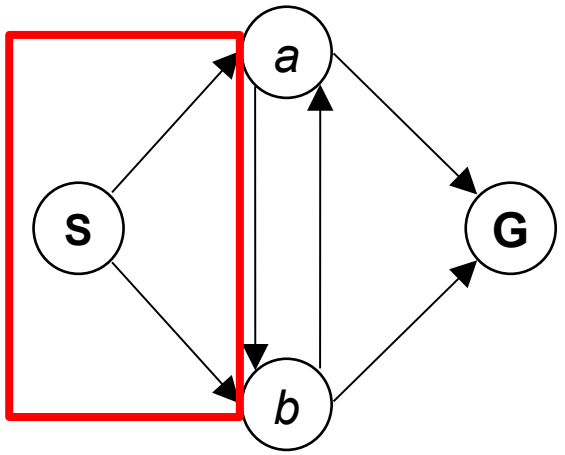


On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>
frontier = {<S,a>}
node = <S>
node.STATE = S
child-state = a
child-node = <S,a>

Consider this 4-state graph:

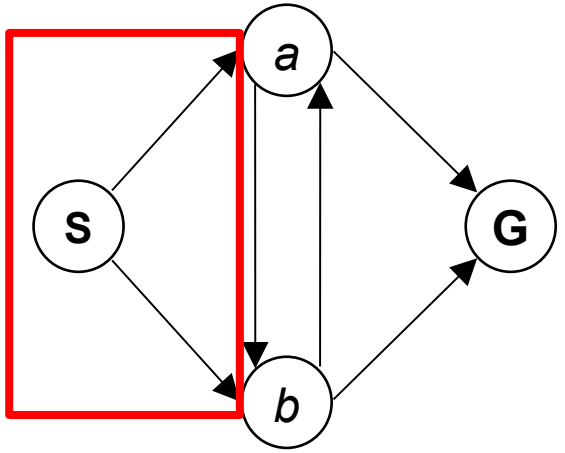


On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>
frontier = {<S,a>}
node = <S>
node.STATE = S
child-state = b

Consider this 4-state graph:

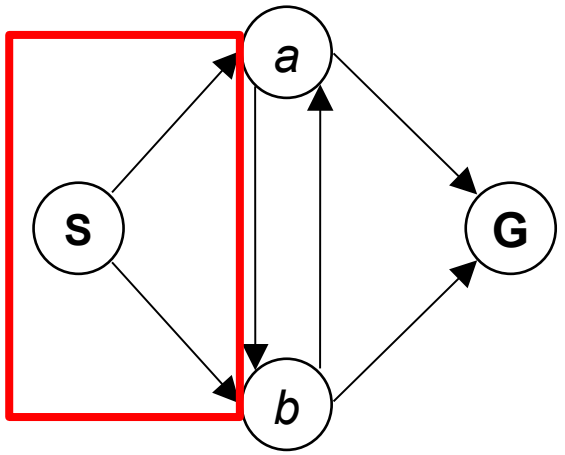


On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>
frontier = {<S,a>}
node = <S>
node.STATE = S
child-state = b
child-node = <S,b>

Consider this 4-state graph:

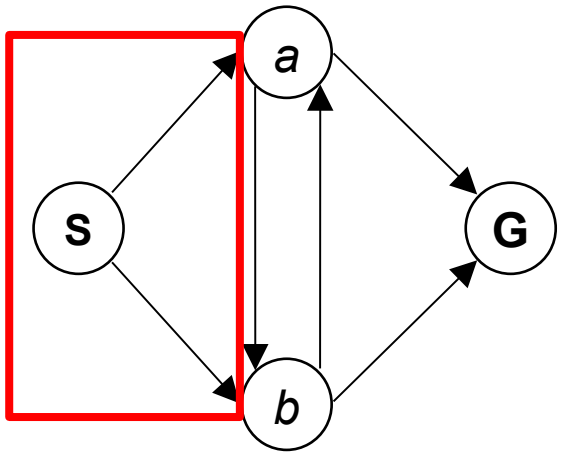


On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>
frontier = {<S,a>,<S,b>}
node = <S>
node.STATE = S
child-state = b
child-node = <S,b>

Consider this 4-state graph:



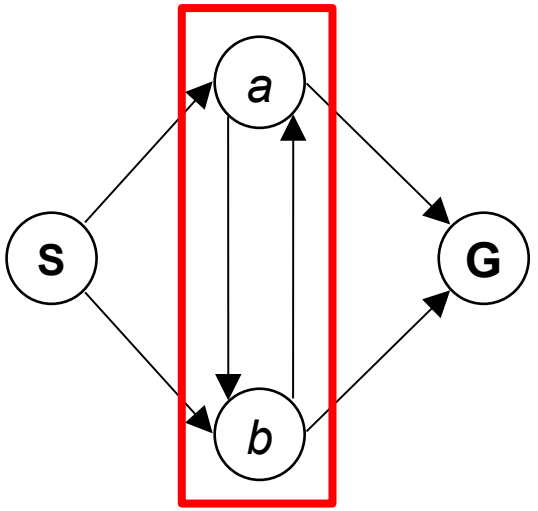
On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>

frontier = {<S,a>,<S,b>}

Consider this 4-state graph:

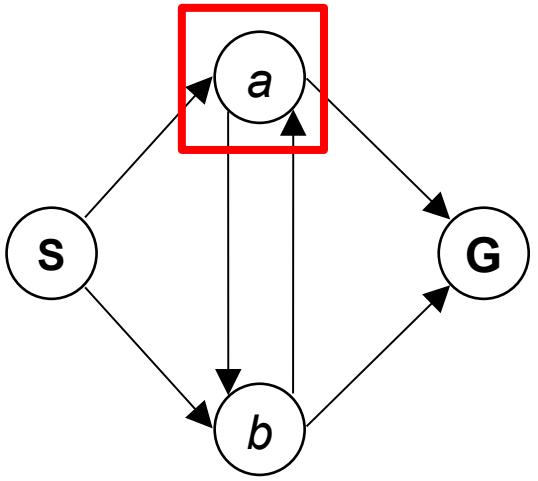


On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>
frontier = {<S,b>}
node = <S,a>

Consider this 4-state graph:

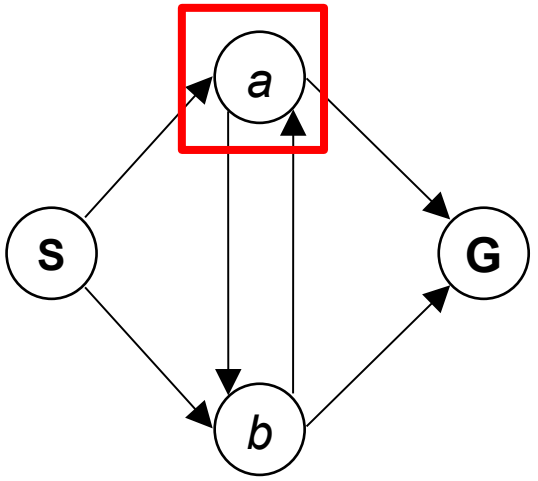


On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>
frontier = {<S,b>}
node = <S,a>
node.STATE = a

Consider this 4-state graph:



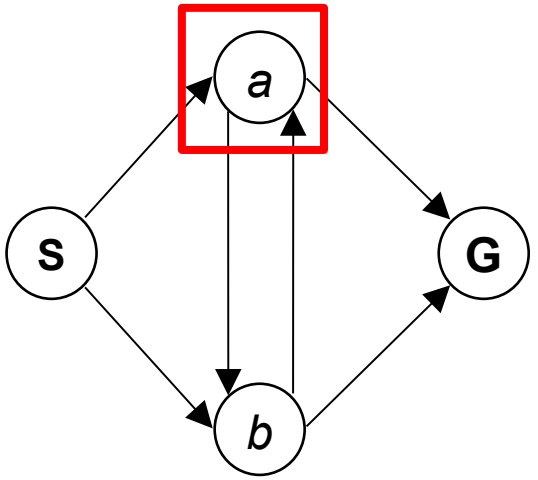
On-demand tree search

```
function TREE-SEARCH(problem)   # return a solution or failure
        start-node := MAKE-NODE(problem.INITIAL-STATE())
        frontier := set consisting of start-node
        while not IS-EMPTY(frontier) do:
                node := select and remove an element from frontier
                if problem.IS-GOAL-STATE(node.STATE) then
                        return node
                for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                        child-node := MAKE-NODE(child-state, node)
                        frontier := INSERT(child-node, frontier)
        return FAILURE
```

start-node = <S>
frontier = {<S,b>}
node = <S,a>
node.STATE = a

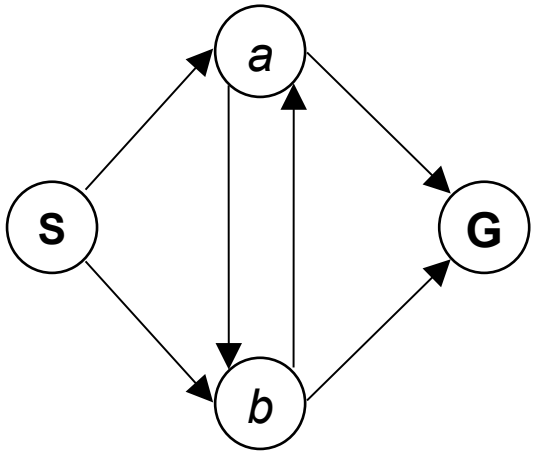
● ● ●

Consider this 4-state graph:



Quiz: State Space Graphs vs. Search Trees

Consider this 4-state graph:

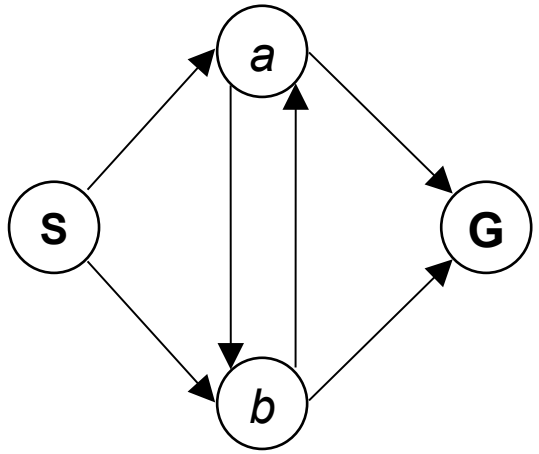


How big is its search tree (from S)?

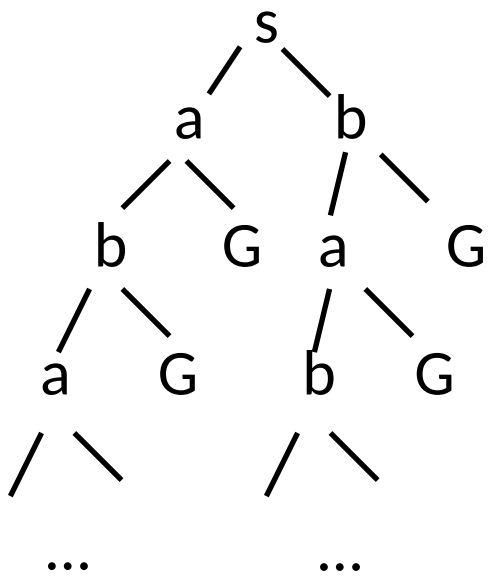


Quiz: State Space Graphs vs. Search Trees

Consider this 4-state graph:



How big is its search tree (from S)?

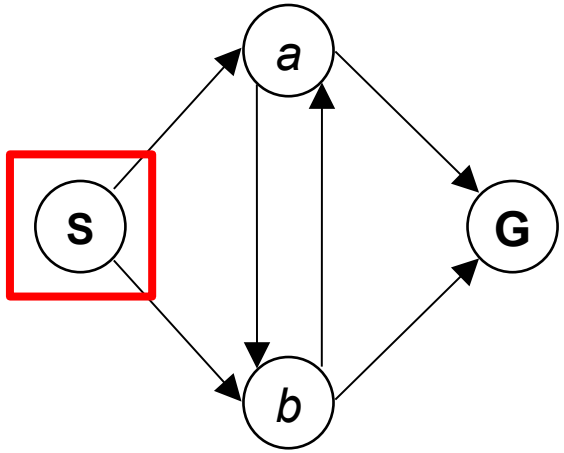




Important: Those who don't know history are doomed to repeat it!

On-demand tree search, with loop detection*

```
function GENERIC-SEARCH(problem)   # return a solution or failure
    start-node := MAKE-NODE(problem.INITIAL-STATE())
    frontier := set consisting of start-node
    closed-states := an empty set
    while not IS-EMPTY(frontier) do:
        node := select and remove an element from frontier
        if problem.IS-GOAL-STATE(node.STATE) then
            return node
        if node.STATE not in closed-states then
            add node.STATE to closed-states
            for child-state in problem.NEIGHBOR-STATES(node.STATE) do:
                child-node := MAKE-NODE(child-state, node)
                frontier := INSERT(child-node, frontier)
    return FAILURE
```

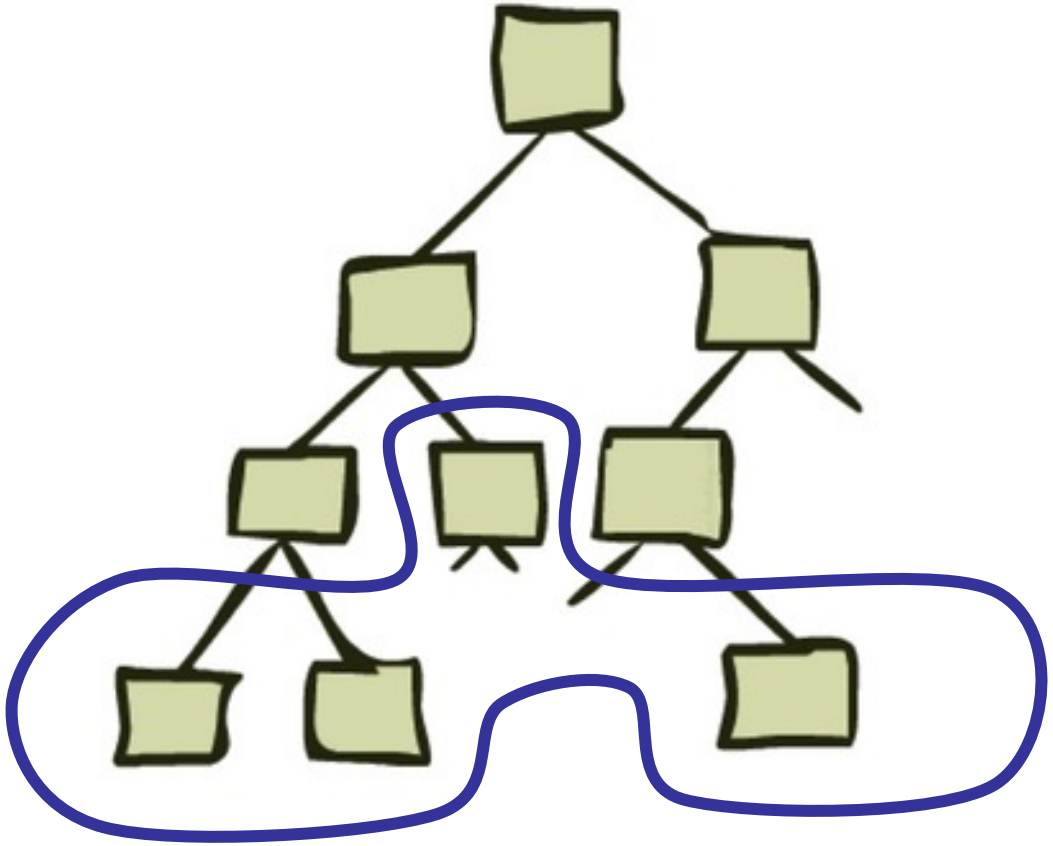


start-node = <S>

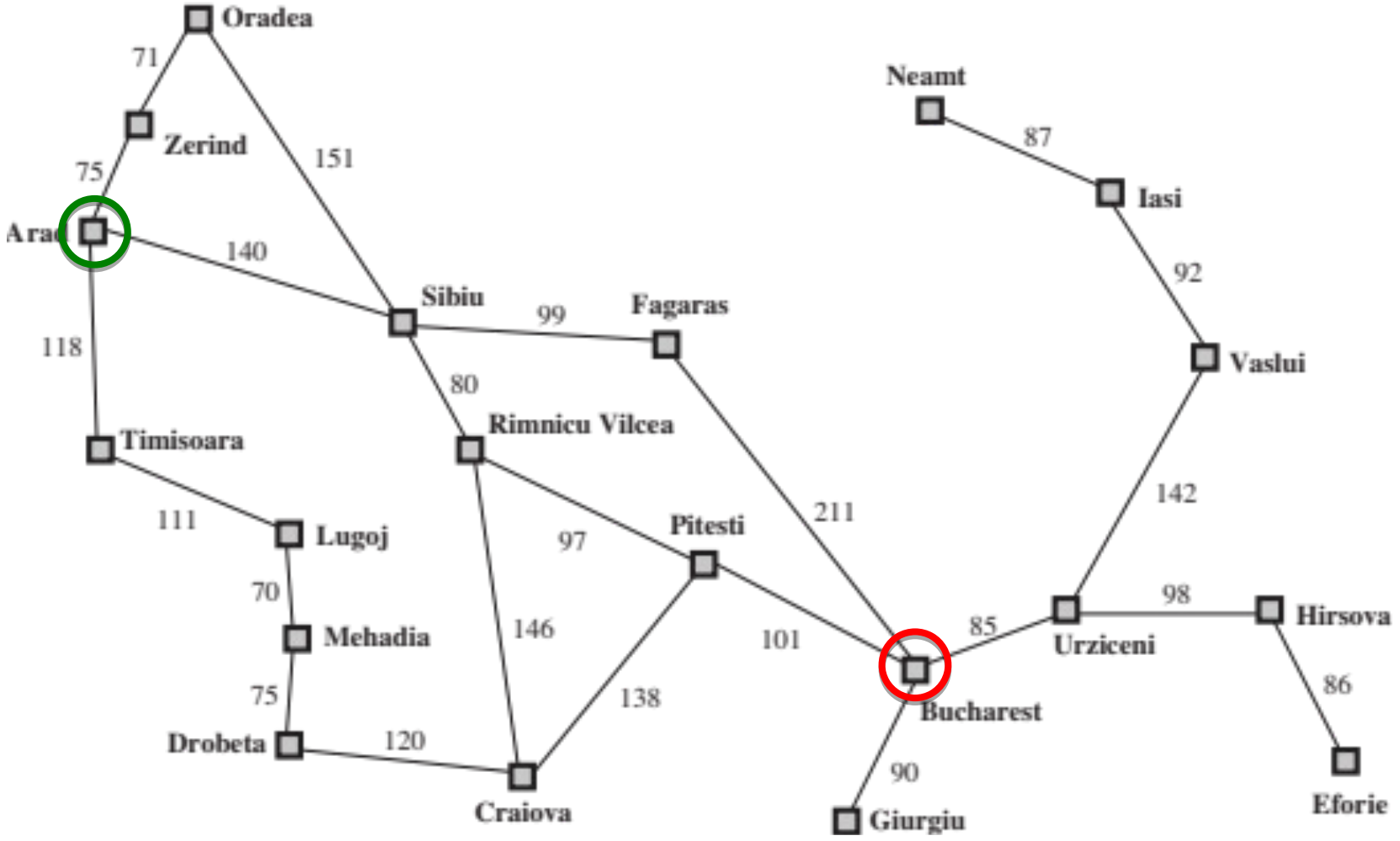
Verify its correctness on this case at home!

** in the CS188 course (and project), they call this graph search because it works on any graph, while tree search only works on trees*
in earlier versions of this course (and Prof. De Raedt's), this would be called 'graph search with late goal checking and closed set loop breaking'

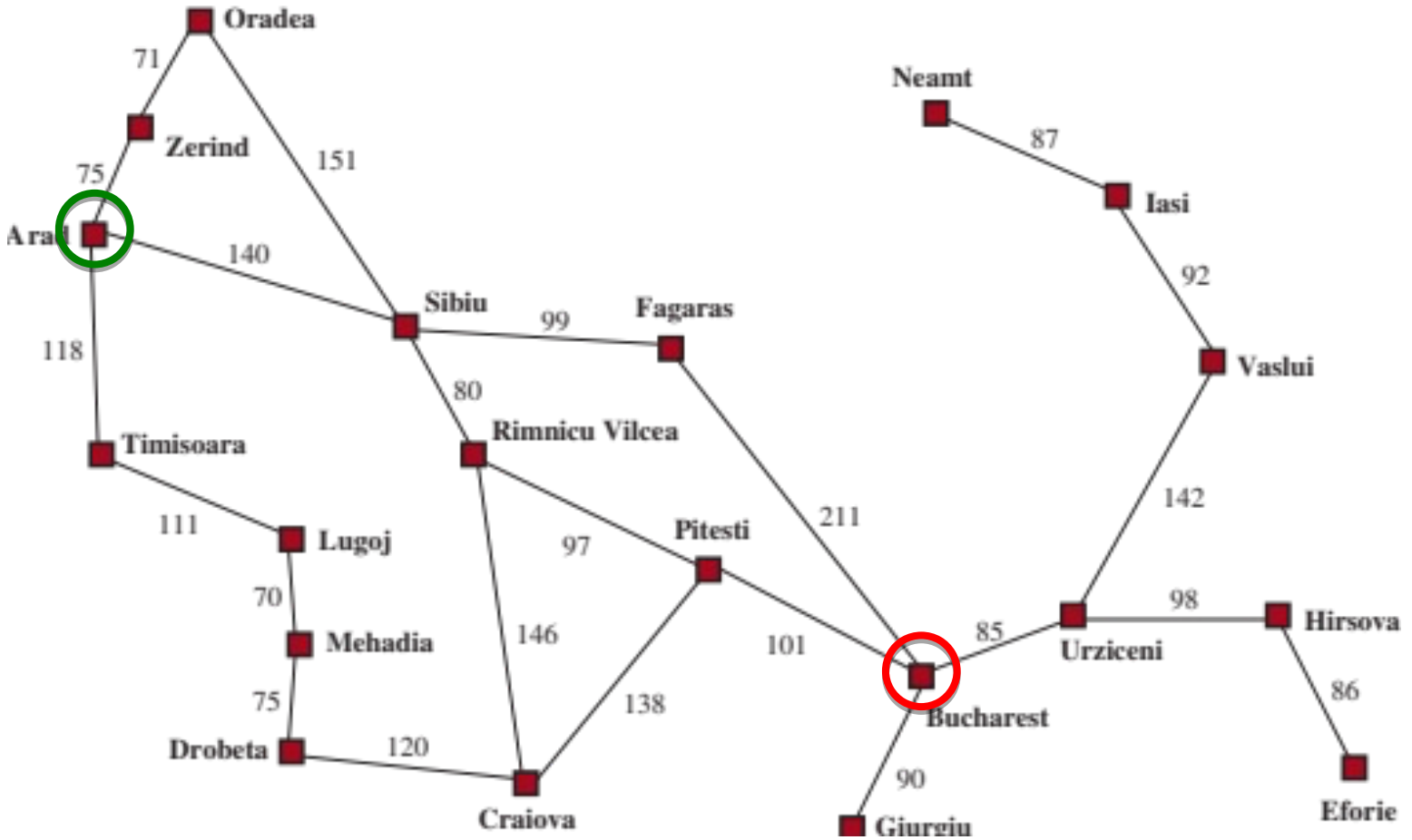
Generic Search



Example: route-finding in Romania



Example: Traveling in Romania

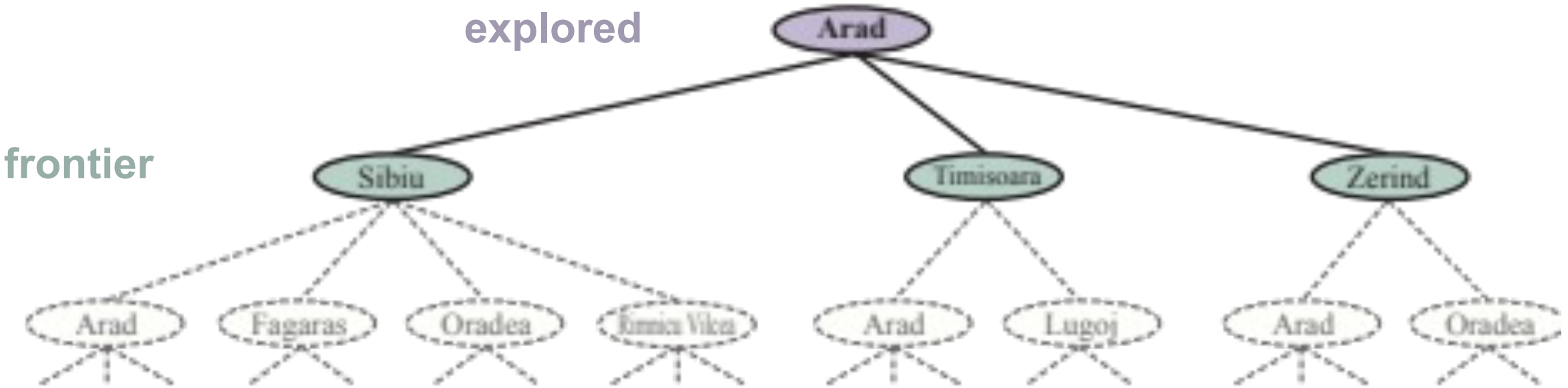


- State space:
  - Cities
- Initial state:
  - Arad
- Actions:
  - Go to adjacent city
- Transition model:
  - Reach adjacent city
- Goal test:
  - *s* = Bucharest?
- Action cost:
  - Road distance from *s* to *s'*
- Solution?

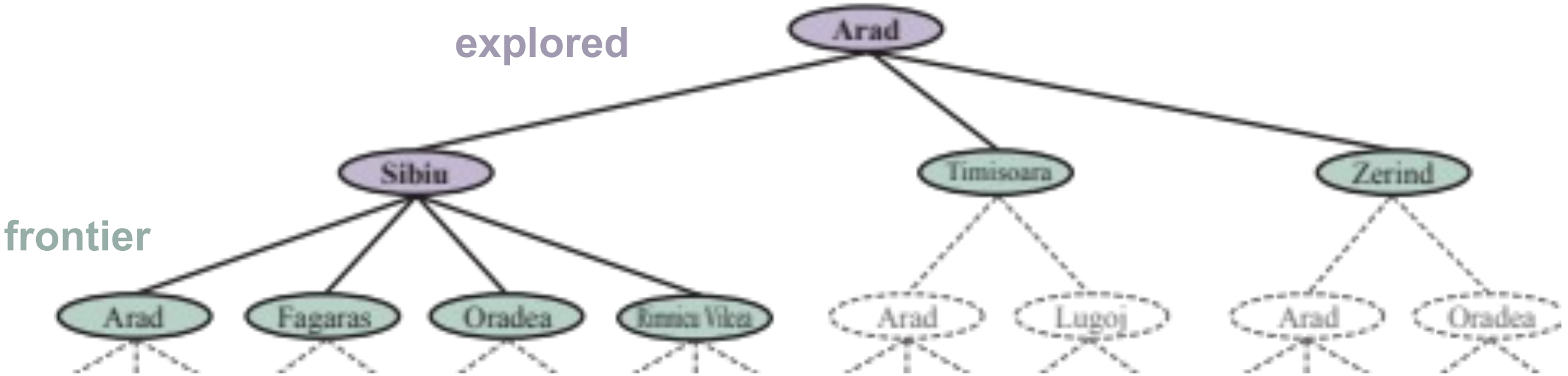
Creating the search tree



Creating the search tree

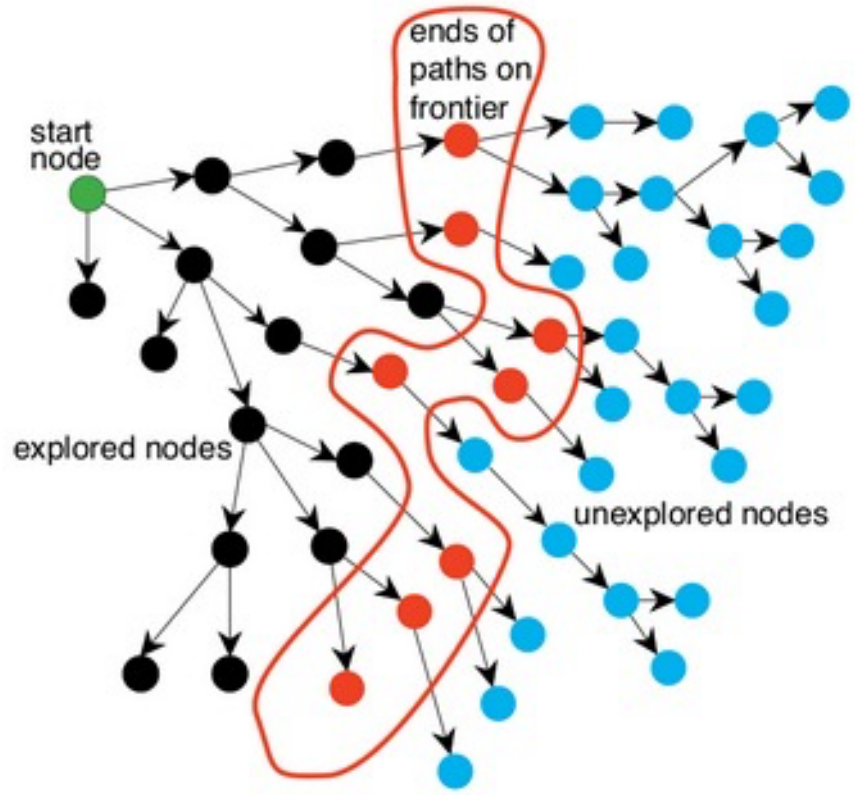


Creating the search tree



Key concepts

- frontier / fringe: set of all nodes available for expansion
- node expansion: generating all successor nodes considering the available actions
- search strategy: defines which node is expanded next
  ^^ main question!



RECAP: Pac-Man as an Agent

